UNIVERSITÀ DEGLI STUDI DI NAPOLI "FEDERICO II"
Scuola Politecnica e Delle Scienze di Base
Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione

DietiEstates25



Progetto di Ingegneria del Software

Martina Mele N86004642 **Simone Del Gais** N86004640

Alessia Marasco N86004600

Indice

Documento dei Requisiti Software **3**

1 Glossario **3**

2 Introduzione **5**

3 Requirement Elicitation and Analysis **7**
3.1 Individuazione del target degli utenti tramite Personas 7

4 Requirement Specification **9**
4.1 Use Case Diagrams 9
4.2 Requisiti Funzionali 9
4.3 Requisiti non Funzionali 11
4.4 Requisiti di Dominio 11
4.5 Rappresentazione Tabellare: Modello di A. Cockburn 12

Documento di Design del Sistema **20**

5 Progettazione Architetturale e Scelte Tecnologiche **20**
5.1 Descrizione dell'Architettura 20
5.2 Criteri di Design e Motivazioni 21
5.3 Tecnologie Adottate 22
5.4 Schema per la Persistenza dei Dati 24
5.4.1 Descrizione delle entità e delle relazioni 24

6 Design dell'Interfaccia Utente **26**
6.1 Colori 26
6.2 Tipografia 27
6.3 Labor Perception Bias 28
6.4 Principles of Proximity and Connecttedness 28
6.5 F-Pattern 30
6.6 Feedback Persistence 30
6.7 Prevenire gli Errori 31

7 Diagrammi delle classi di Design **33**
7.1 Classi di Design del Backend 33
7.1.1 Architettura Controller-Service-Repository 33
7.1.2 DTO 33
7.1.3 Utility, Config e Security 34
7.1.4 Entità di Dominio (Model) 34
7.2 Classi di Design del Frontend 36
7.3 Diagrammi di Sequenza di Design per gli Use Case 37

Documentazione del Processo di Sviluppo **39**

8 File di build automatica **39**
8.1 DockerFile (Backend) . . . 39
8.2 DockerFile (Frontend) . . . 39
8.3 Docker Compose . . . 39

9 Strumento di Versioning **40**

10 Qualità del codice **41**
10.1 Grafico di Duplicazione del Codice . . . 42
10.2 Grafico delle Issues . . . 42
10.3 Grafico della Coverage dei Test . . . 43

Testing e Valutazione dell'Usabilità **44**

11 Unit Testing **44**
11.1 Test di loadNewRealEstate (EstateAgentService) . . . 44
11.2 Test di bookVisit (BuyerService) . . . 48
11.3 Test di registerBuyerWithGoogle (BuyerService) . . . 51
11.4 Test di getWeatherForecast (WeatherApiService) . . . 54
11.5 Strategie adottate . . . 57
11.5.1 Controller . . . 57
11.5.2 Service . . . 58
11.5.3 Repository . . . 58
11.5.4 Security e Utils . . . 59

12 Valutazione dell'Usabilità **61**
12.1 Expert Review . . . 61
12.1.1 Obiettivo del Test . . . 61
12.1.2 Istruzioni per l'Uso della Checklist . . . 61
12.1.3 Risultati . . . 62
12.2 Relazione Sperimentale . . . 63
12.2.1 Descrizione dei Partecipanti . . . 63
12.2.2 Procedura Sperimentale . . . 63
12.2.3 Dati Raccolti . . . 64
12.2.4 Valutazione post-task . . . 65
12.2.5 Conclusioni . . . 69

1 Glossario

- **Agente Immobiliare**
  Utente professionale che inserisce e gestisce gli annunci di immobili, risponde alle richieste dei clienti e organizza le visite agli immobili.

- **Amministratore/Admin Agenzia Immobiliare**
  Figura che gestisce la piattaforma a livello organizzativo e ridistribuisce il lavoro tra agenti immobiliari e collaboratori.

- **API (Application Programming Interface)**
  Insieme di regole e protocolli che consentono a diversi software di comunicare tra loro, definendo i metodi e i formati di dati che possono essere utilizzati per lo scambio di informazioni.

- **API REST-ful**
  Tipo di API basate sui principi REST (Representational State Transfer), che utilizzano il protocollo HTTP per gestire le operazioni di creazione, lettura, aggiornamento ed eliminazione dei dati (CRUD) attraverso metodi standard come POST, GET, PUT e DELETE.

- **Architetture Three Tier**
  Modello architetturale di software suddiviso in tre livelli distinti: presentazione, logica applicativa e dati.

- **DTO (Data Transfer Object)**
  Oggetto progettato per trasferire dati tra differenti strati di un'applicazione (ad esempio tra il backend e il frontend) evitando di esporre direttamente le entità del database.

- **Framework**
  Struttura software predefinita che fornisce strumenti, librerie e linee guida per semplificare e velocizzare lo sviluppo di applicazioni, garantendo coerenza e organizzazione nel codice.

- **JSON (JavaScript Object Notation)**
  Formato di interscambio dati testuale, leggero e facilmente leggibile, basato su coppie "chiave: valore", utilizzato principalmente per la comunicazione tra client e server.

- **Otto Regole d'Oro di Shneiderman**
  Insieme di principi per la progettazione di interfacce utente efficaci: coerenza nel design, disponibilità di scorciatoie, feedback informativo, chiusura del dialogo, prevenzione e correzione degli errori, reversibilità delle azioni, controllo da parte dell'utente e riduzione del carico cognitivo.

- **Policy CORS (Cross-Origin Resource Sharing)**
  Regola di sicurezza che determina se un dominio esterno può accedere alle risorse di un sito web, proteggendo così da richieste non autorizzate provenienti da origini differenti.

- **Richieste HTTP**
  Messaggi inviati da un client a un server per ottenere o inviare informazioni.

- **Ricerche Salvate nei Preferiti**
  Funzionalità che consente all'utente di memorizzare determinati criteri di ricerca, in modo da poterli recuperare facilmente in momenti successivi.

- **Schema UML (Unified Modeling Language)**
  Linguaggio standard per rappresentare graficamente la struttura e il comportamento di un sistema software attraverso diagrammi, come quelli delle classi, dei casi d'uso o di sequenza.

- **Sistema ORM (Object-Relational Mapping)**
  Tecnica che consente di interagire con un database relazionale utilizzando oggetti del linguaggio di programmazione, trasformando automaticamente le tabelle in classi e i record in istanze di oggetti.

- **Single Page Application (SPA)**
  Applicazione web che carica una sola pagina HTML e aggiorna dinamicamente i contenuti al suo interno.

- **Test R-WECT** (Reading, Writing, Error, Consistency, Time)
  Una metodologia di verifica che analizza la qualità e l'efficienza di un sistema, valutando aspetti come la leggibilità, la gestione degli errori, la coerenza e i tempi di risposta.

- **Token**
  Stringa univoca utilizzata per autenticare un utente o autorizzare l'accesso a risorse protette.

- **Utente Privato in Cerca di Immobili**
  Persona che utilizza la piattaforma per cercare e visualizzare gli immobili, e, in caso, contattare gli agenti per fissare visite.

- **WCAG (Web Content Accessibility Guidelines)**
  Insieme di linee guida internazionali per rendere i contenuti web accessibili a tutti gli utenti, comprese le persone con disabilità. Le WCAG stabiliscono principi e criteri per garantire percepibilità, operabilità, comprensibilità e robustezza delle interfacce digitali.

2 Introduzione

L'obiettivo di questo documento è fornire una definizione, attraverso diversi strumenti di formalizzazione, delle funzionalità del sistema che si andrà a sviluppare. Il prodotto finale è una piattaforma per la gestione di servizi immobiliari: DietiEstates25.

Le principali funzionalità richieste sono:

- Ogni installazione di DietiEstates25 include un'utenza di amministrazione per il gestore dell'agenzia immobiliare, che viene creata con credenziali predefinite. Dopo aver effettuato l'accesso, è possibile modificare la password di amministrazione. L'amministratore può creare altri account di supporto all'amministrazione. I gestori di agenzie immobiliari possono a loro volta creare account per gli agenti immobiliari. Un utente interessato ai servizi immobiliari può registrarsi con un'e-mail e una password. È apprezzata la possibilità di effettuare la registrazione/login tramite credenziali di terze parti (e.g.: accedi con account Google, Facebook o GitHub). Gli utenti possono effettuare il login e accedere all'applicativo. Le credenziali devono essere salvate in modo sicuro.

- Gli agenti immobiliari possono caricare nuovi immobili con una serie di dettagli: foto, descrizione, prezzo, dimensioni, indirizzo, numero di stanze, piano, presenza di ascensore, classe energetica, ulteriori servizi presenti (portineria, climatizzazione, ...), ecc. Gli immobili devono essere categorizzati in "vendita", "affitto". Forse in futuro sarà introdotto il supporto per affitti brevi e "case vacanze" (ma questa funzionalità al momento non è prevista). Ogni immobile inoltre è associato alla posizione geografica esatta, individuata interagendo con una mappa interattiva (tipo Google Maps).

- Gli utenti possono eseguire una ricerca avanzata di immobili con parametri multipli: tipologia di inserzione, prezzo minimo e massimo, numero di stanze, classe energetica, posizione, ecc. In particolare, deve essere possibile effettuare filtraggi per posizione geografica almeno al livello di comune o città. È apprezzata –ma non obbligatoria – la possibilità di effettuare ricerche di immobili che si trovano in un raggio arbitrario da un certo punto specificato dall'utente e selezionato su una mappa interattiva. La ricerca deve essere efficiente e performante. Gli immobili individuati nelle ricerche vengono visualizzati su una mappa interattiva (tipo Google Maps).

- Gli utenti devono poter modificare i loro dati personali: nome, cognome, e-mail, password. Gli agenti immobiliari devono avere un profilo più dettagliato, con informazioni professionali (e.g.: una breve biografia) e magari una valutazione lasciata dai clienti. È gradita la possibilità di aggiungere anche delle foto per gli agenti immobiliari, per rendere i profili più "umani"

- I clienti devono poter prenotare una visita (nelle successive due settimane) per un immobile direttamente dal sistema. Gli agenti immobiliari ricevono notifiche (e-mail o interne al sistema, o possibilmente entrambe) quando una visita è prenotata per uno dei loro immobili. Le visite possono essere confermate o rifiutate dagli agenti. Se un appuntamento viene rifiutato, il cliente deve poter selezionare un altro orario.

Sarebbe apprezzata la possibilità di integrare un calendario per gestire visivamente gli appuntamenti di un agente immobiliare.

- Quando prenotano una visita, gli utenti visualizzano anche le previsioni meteo per i diversi giorni in cui è possibile prenotare, in modo da poter pianificare meglio la visita secondo le proprie preferenze e necessità. Le previsioni meteo vanno ottenute tramite API di servizi esterni come Open Meteo.
- Il sistema deve tenere traccia delle ricerche effettuate da un utente in precedenza, in modo tale che un utente possa facilmente ri-eseguire interrogazioni già eseguite in passato per vedere se sono presenti nuove inserzioni che soddisfano le caratteristiche richieste.
- Il sistema deve inviare notifiche agli utenti per vari eventi: nuove proprietà in linea con le loro ricerche precedenti, conferma o rifiuto delle visite, messaggi promozionali.Gli utenti devono poter attivare/disattivare singole categorie di notifiche.

3 Requirement Elicitation and Analysis

3.1 Individuazione del target degli utenti tramite Personas

Gli utenti principali del sistema sono di tre tipologie: utenti privati in cerca di un immobile, amministratori di agenzie immobiliari e agenti immobiliari.
Utilizzando le Personas, ovvero degli archetipi di utenti target del sistema, possiamo meglio calarci nei loro panni e capire quali sono i loro bisogni e integrarli nel nostro sistema.

Studiando i bisogni degli utenti tipo notiamo che all'interno dell'agenzia le diverse figure hanno necessità quasi opposte tra loro.
Un amministratore ha moltissimi compiti eterogenei e quelli che concernono il nostro applicativo sono una piccolissima parte. Le funzionalità da implementare sono poche, ma bisogna che siano il più intuitive e semplici possibili, senza rinunciare alla sicurezza.

Per un agente immobiliare le possibilità di aggiungere immobili, creare annunci e interagire con gli utenti interessati sono fondamentali per il lavoro di tutti i giorni. Per questo motivo è cruciale ridurre il tempo e l'impegno necessari per le operazioni più ripetitive.

Gli utenti interessati agli immobili si impegneranno nella ricerca, ed è importante fornire un'interfaccia che li assista in questo, ma è altrettanto fondamentale aiutarli anche quando non possono essere attivi, con notifiche personalizzate in base alle loro preferenze.

Al Goritmo



Info

Anni:	29
Genere:	Uomo
Stato Familiare:	Celibe
Figli:	Nessuno
Professione:	Infermiere

Background

Al lavora da tre anni come infermiere, e dopo aver messo da parte dei risparmi restando a casa dei genitori, ha finalmente deciso di trovare una casa in affitto tutta sua.

Obiettvi

- Trovare un appartamento bilocale con un budget limitato;
- Risparmiare tempo nella ricerca;
- Ricevere notifiche su nuove inserzioni pertinenti alle sue preferenze.

Privato in cerca di un immobile

Punti di forza

- Buona padronanza delle tecnologie;
- Precisione e attenzione ai dettagli.

Debolezze

- Elevata aspettativa di personalizzazione e qualità di ricerca;
- Facilmente frustrato se la ricerca non è intuitiva o efficace.

Spunti

- Ricerca avanzata degli immobili;
- Storico ricerche;
- Notifiche personalizzate;
- Interfaccia user-friendly.

Lina Guaggio



Info

Anni:	50
Genere:	Donna
Stato Familiare:	Coniugata
Figli:	Due
Professione:	Imprenditrice

Background

Lina gestisce un'agenzia immobiliare a conduzione familiare da quindici anni. Le sue mansioni sono incentrate su coordinamento del personale, contabilità e conformità legale.
I compiti e le responsabilità sono tante e il tempo è poco.

Obiettvi

- Semplificare gli obblighi relativi alla gestione del personale;
- Garantire che il team utilizzi strumenti moderni, efficienti e sicuri.

Amministratrice di un'agenzia

Punti di forza

- Buona conoscenza del settore;
- Approccio pragmatico.

Debolezze

- Poca familiarità con piattaforme digitali complesse;
- Gran parte del suo lavoro non è svolta sull'applicativo;
- Poca conoscenza delle mansioni operative.

Spunti

- Interfaccia semplice e intuitiva;
- Steep learning curve;
- Accesso rapido a tutte le funzionalità essenziali, escludendo quelle operative che non le concernono.

Ada Pter



Info

Anni:	28
Genere:	Donna
Stato Familiare:	Nubile
Figli:	Nessuno
Professione:	Agente immobiliare

Background

Ada è un agente immobiliare junior che lavora per una grande agenzia con decine di immobili in gestione. È dinamica e abituata a utilizzare strumenti digitali per il lavoro. Passa la maggior parte del tempo fuori l'ufficio, tra appuntamenti con clienti e visite agli immobili.

Obiettvi

- Caricare annunci rapidamente;
- Ridurre il tempo necessario per compiti ripetitivi;
- Tener traccia degli appuntamenti con i clienti.

Agente immobiliare junior

Punti di forza

- Familiarità con le tecnologie;
- Capacità di adattamento.

Debolezze

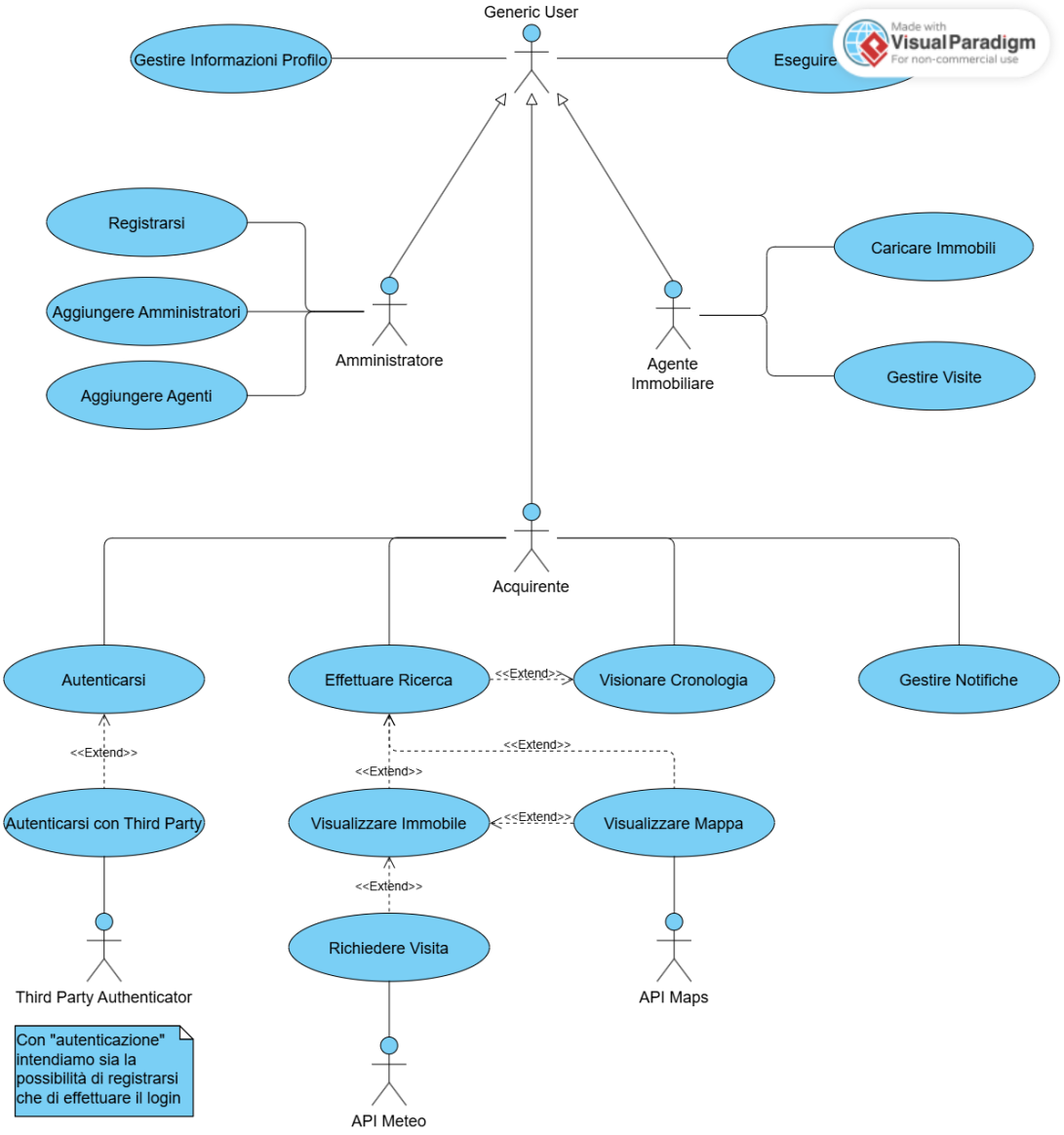
- Può distrarsi facilmente o dimenticare qualcosa a causa della giornata lavorativa frenetica.

Spunti

- Semplificare la gestione degli appuntamenti;
- Ottimizzare la creazione di annunci;
- Possibilità di ricevere notifiche.

4 Requirement Specification

4.1 Use Case Diagrams



4.2 Requisiti Funzionali

I requisiti funzionali sono i servizi che il sistema offre agli utenti.

DENOMINAZIONE	DESCRIZIONE
Registrazione Utente	Un utente interessato a visionare degli immobili può creare un proprio account.
Registrazione Utente con Credenziali di Terze Parti	Un utente può creare un account per DietiEstates25 utilizzando credenziali già esistenti di terze parti.

Login Utente	Un utente che abbia già effettuato la registrazione può accedere al suo personale account e ai servizi.
Login Utente con Credenziali di Terze Parti	Un utente che abbia già effettuato la registrazione può accedere al suo personale account e ai servizi utilizzando credenziali già esistenti di terze parti.
Ricerca Immobile	Un utente può ricercare gi immobili presenti nel sistema tramite una parola chiave
Ricerca Immobile Avanzata	Un utente può filtrare i risultati di ricerca in base alle proprie preferenze.
Visiualizzazione Inserzione	Un utente che sia interessato a un annuncio, cliccando sull'anteprima, può visionarlo completamente.
Richiesta Visita	Un utente interessato a visitare un immobile può fare richiesta per una data nelle successive due settimane.
Interazione con la Cronologia	Un utente può visionare la sua cronologia di ricerca e ri-effettuare le interrogazioni.
Gestione Informazioni Profilo	Un utente, un admin o un agente può visualizzare le informazioni del proprio profilo e modificarle.
Registrazione Amministratore	Un amministratore di un'agenzia immobiliare può richiedere delle credenziali predefinite per accedere al sistema.
Login Area Agenzia	Un amministratore o un agente immobiliare che abbiano già un account, devono necessariamente, solo per il primo accesso, modificare le proprie credenziali, per poi accedere al proprio account.
Aggiunta Amministratore	Un amministratore può creare un account di supporto che abbia la sua stessa autorità.
Aggiunta Agenti Immobiliari	Un amministratore può creare un account con credenziali predefinite per gli agenti immobiliari
Creazione Inserzione Immobiliare	Un agente immobiliare può inserire un annuncio di un immobile, allegato di immagini e diverse informazioni relative a posizione, dettagli strutturali e offerta.
Notifiche	In base alle proprie preferenze, un utente riceve notifiche per vari eventi. Inoltre utenti e agenti possono interagire per accordarsi sulle visite

4.3 Requisiti non Funzionali

Quelli che seguono sono vincoli sui servizi offerti dall'apllicativo.

DENOMINAZIONE	DESCRIZIONE
Univocità Email	Può esistere un solo account nel sistema connesso a una determinata mail.
Modifica delle Credenziali Predefinite	Quando un account dell'area agenzia esegue il primo login con delle credenziali predefinite, l'utente deve necessariamente modificarle prima di poter eseguire il logout.
Sicurezza delle Credenziali	Il sistema deve garantire una gestione sicura di tutte le credenziali, sia informazioni personali che password.
Ricerca Efficiente e Performante	La ricerca avanzata di immobili deve essere eseguita in modo rapido ed efficiente, anche con grandi volumi di dati.

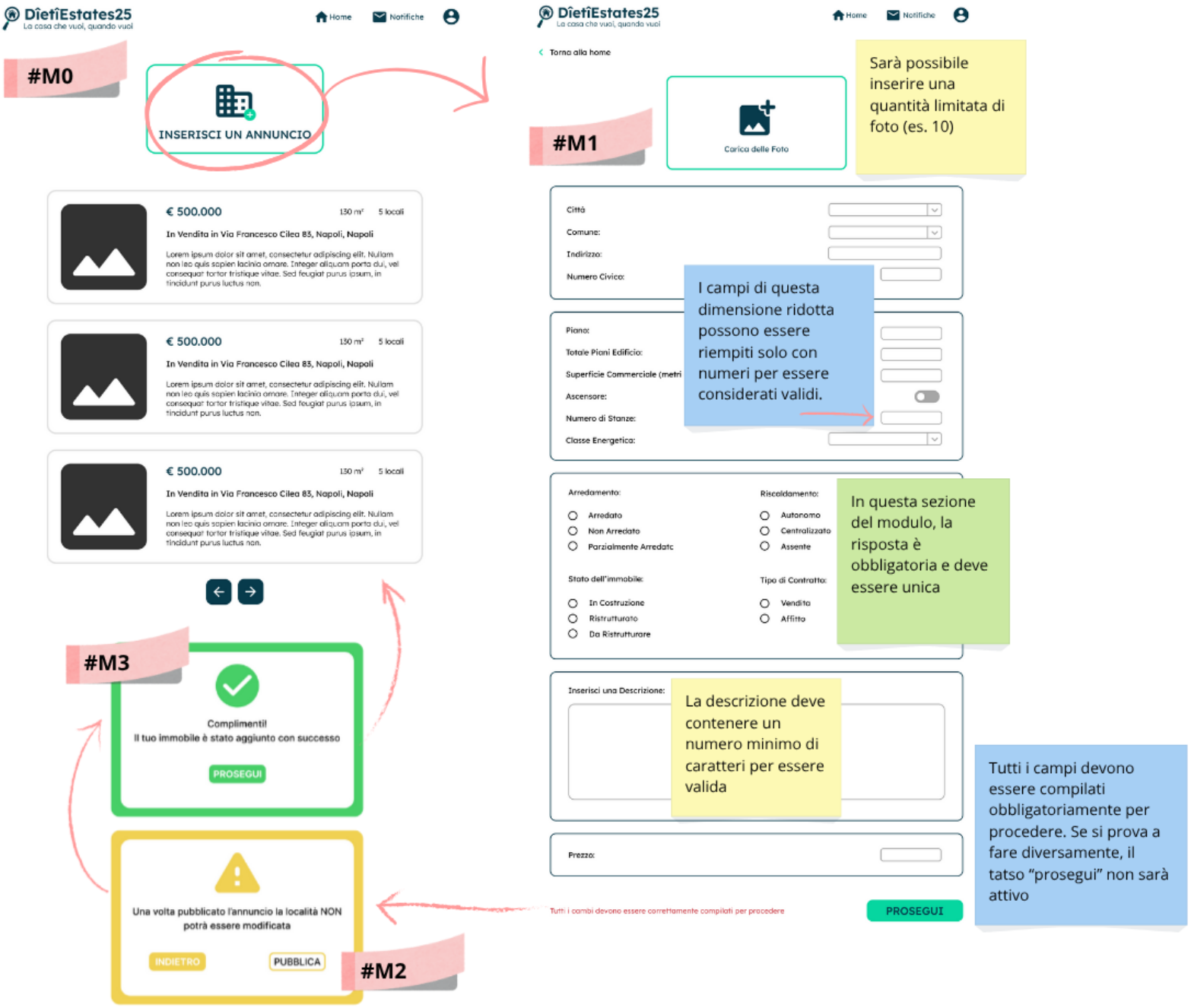
4.4 Requisiti di Dominio

I requisiti di dominio sono dei vincoli dell'applicazione che derivano dal contesto specifico del dominio immobiliare.

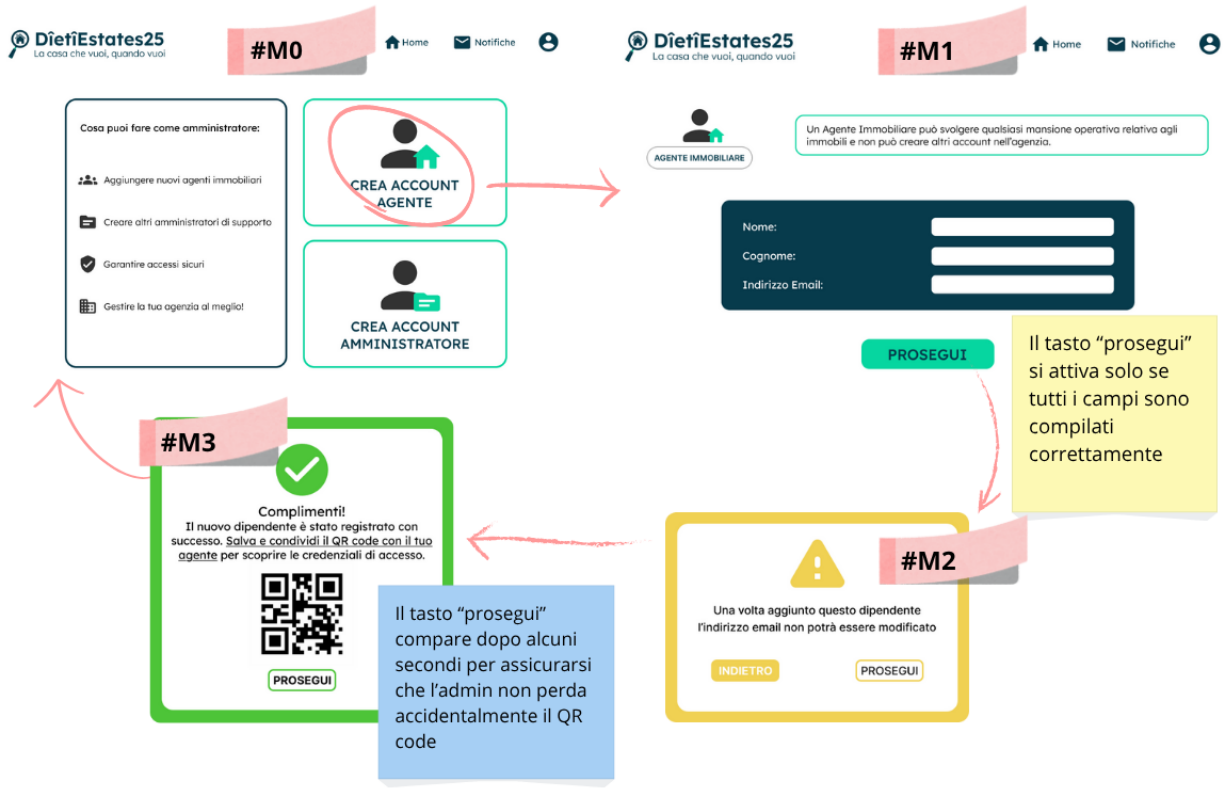
DENOMINAZIONE	DESCRIZIONE
Indipendenza di Back-end e Front-end	Bisogna implementare un sistema distribuito in cui le due parti devono poter essere eseguite senza alcuna dipendenza dall'altra.
Facilità di Estensione	Il sistema deve essere progettato con una architettura modulare e ben documentata per agevolare l'aggiunta di nuove funzionalità o modifiche future senza compromettere le prestazioni o la stabilità del sistema.
Dettagli Specifici per gli Immobili	Ogni immobile deve includere informazioni dettagliate richieste dal settore immobiliare, come classe energetica, presenza di ascensore, portineria, climatizzazione, ecc.
Supporto alla Geolocalizzazione	Gli immobili devono essere associati a una posizione geografica precisa e le ricerche devono supportare filtri per posizione geografica e raggio di distanza.
Univocità Partita IVA	Può esistere una sola agenzia immobiliare nel sistema connessa a una determinata partita IVA.

4.5 Rappresentazione Tabellare: Modello di A. Cockburn

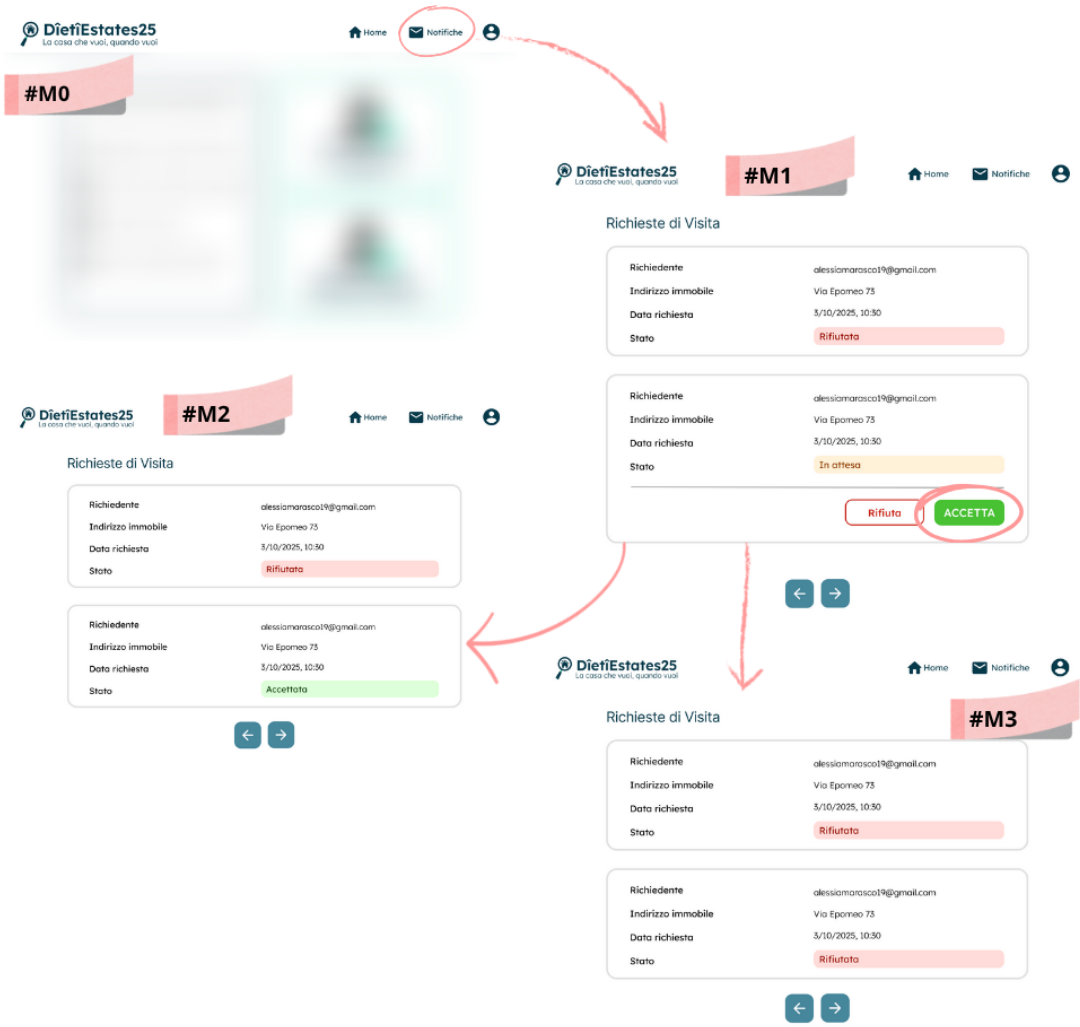
USE CASE#1	Agente Immobiliare Carica un Immobile		
Goal in Context	Un agente immobiliare vuole caricare un annuncio di un immobile		
Preconditions	L'agente immobiliare deve aver effettuato il caso d'uso "Login"		
Success End Condition	Il Sistema tiene traccia del nuovo immobile; l'annuncio è visibile a qualsiasi utente interessato		
Failed End Condition	Il Sistema non tiene traccia dell'immobile; l'annuncio non risulta essere caricato		
Primary Actor	Agente Immobiliare		
Trigger	Agente Immobiliare clicca "Aggiungi un Nuovo Immobile" in #M0		
Main Scenario	**Step n.**	**Agente**	**Sistema**
	1	Clicca "Aggiungi un Nuovo Immobile" in #M0	
	2		Mostra #M1 (con "PROSEGUI" disattivato)
	3	Compila correttamente e interamente il form	
	4		Attiva il pulsante "PROSEGUI"
	5	Clicca il pulsante "PROSEGUI"	
	6		Mostra #M2
	7	Clicca il pulasnte "PUBBLICA"	
	8		Mostra #M3"
	9	Clicca il pulasnte "PROSEGUI"	
	10		Mostra #M0 e termina UC
Extension A: Clicca freccia "indietro" in #M1	3a	Clicca freccia "indietro"	
	4a		Mostra #M0 e termina UC
Extension B: Clicca il pulsante "INDIETRO" sulla finestra di conferma	7b	Clicca il pulsante "INDIETRO"	
	8b		Ritorna allo step 4


DîetîEstates25
La casa che vuoi, quando vuoi
Home
Notifiche
#M0
INSERISCI UN ANNUNCIO
€ 500.000
130 m²
5 locali
In Vendita in Via Francesco Cilea 83, Napoli, Napoli
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam non leo quis sapien lacinia ornare. Integer aliquam porta dui, vel consequat tortor tristique vitae. Sed feugiat purus ipsum, in tincidunt purus luctus non.
#M3
Complimenti!
Il tuo immobile è stato aggiunto con successo
PROSEGUI
Una volta pubblicato l'annuncio la località NON potrà essere modificata
INDIETRO
PUBBLICA
#M2
Torna alla home
#M1
Carica delle Foto
Sarà possibile inserire una quantità limitata di foto (es. 10)
Città
Comune:
Indirizzo:
Numero Civico:
Piano:
Totale Piani Edificio:
Superficie Commerciale (metri
Ascensore:
Numero di Stanze:
Classe Energetica:
I campi di questa dimensione ridotta possono essere riempiti solo con numeri per essere considerati validi.
Arredamento:
Arredato
Non Arredato
Parzialmente Arredato
Riscaldamento:
Autonomo
Centralizzato
Assente
Stato dell'immobile:
In Costruzione
Ristrutturato
Da Ristrutturare
Tipo di Contratto:
Vendita
Affitto
In questa sezione del modulo, la risposta è obbligatoria e deve essere unica
Inserisci una Descrizione:
La descrizione deve contenere un numero minimo di caratteri per essere valida
Prezzo:
Tutti i campi devono essere correttamente compilati per procedere
PROSEGUI
Tutti i campi devono essere compilati obbligatoriamente per procedere. Se si prova a fare diversamente, il tatso "prosegui" non sarà attivo

USE CASE#2	Amministratore Aggiunge un Agente		
Goal in Context	L'amministratore di un'agenzia vuole creare un account per un nuovo agente immobiliare		
Preconditions	L'amministratore deve aver effettuato il caso d'uso "Eseguire Login"		
Success End Condition	Il Sistema tiene traccia del nuovo account		
Failed End Condition	Il Sistema non tiene traccia del nuovo account		
Primary Actor	Amministratore		
Trigger	Amministratore clicca "Crea Account Agente" in #M0		
Main Scenario	**Step n.**	**Amministratore**	**Sistema**
	1	Clicca "Crea Account Agente" in #M0	
	2		Mostra #M1 (con "PROSEGUI" disattivato)
	3	Compila correttamente e interamente il form	
	4		Attiva il pulsante "PROSEGUI"
	5	Clicca il pulsante "PROSEGUI"	
	6		Mostra #M2
	7	Clicca il pulsante "PROSEGUI"	
	8		Mostra #M3 (con "PROSEGUI" disattivato)
	9	Salva il QR code	
	10		Attiva "PROSEGUI"
	11	Clicca il pulsante "PROSEGUI"	
	12		Mostra #M0 e termina UC
Extension A: Clicca freccia "indietro" in #M1	3a	Clicca freccia "indietro"	
	4a		Mostra #M0 e termina UC
Extension B: Clicca il pulsante "INDIETRO" sulla finestra di conferma	9c	Clicca il pulsante "INDIETRO"	
	10c		Ritorna allo step 2


DietiEstates25
La casa che vuoi, quando vuoi
#M0
Home
Notifiche
Cosa puoi fare come amministratore:
Aggiungere nuovi agenti immobiliari
Creare altri amministratori di supporto
Garantire accessi sicuri
Gestire la tua agenzia al meglio!
CREA ACCOUNT AGENTE
CREA ACCOUNT AMMINISTRATORE
DietiEstates25
La casa che vuoi, quando vuoi
#M1
Home
Notifiche
AGENTE IMMOBILIARE
Un Agente Immobiliare può svolgere qualsiasi mansione operativa relativa agli immobili e non può creare altri account nell'agenzia.
Nome:
Cognome:
Indirizzo Email:
PROSEGUI
Il tasto "prosegui" si attiva solo se tutti i campi sono compilati correttamente
#M2
Una volta aggiunto questo dipendente l'indirizzo email non potrà essere modificato
INDIETRO
PROSEGUI
#M3
Complimenti!
Il nuovo dipendente è stato registrato con successo. Salva e condividi il QR code con il tuo agente per scoprire le credenziali di accesso.
PROSEGUI
Il tasto "prosegui" compare dopo alcuni secondi per assicurarsi che l'admin non perda accidentalmente il QR code

USE CASE#3	Agente Immobiliare Gestisce Visite		
Goal in Context	Un agente immobiliare vuole gestire una richiesta di visita a un immobile		
Preconditions	L'agente immobiliare deve aver effettuato il caso d'uso "Eseguire Login"		
Success End Condition	Il Sistema tiene traccia della decisione dell'agente e notifica l'utente		
Failed End Condition	Il Sistema non tiene traccia della decisione o non notifica l'utente		
Primary Actor	Agente Immobiliare		
Trigger	Agente Immobiliare clicca "Notifiche" nel menù in alto a destra in una qualsiasi delle pagine, come in #M0		
Main Scenario	**Step n.**	**Agente**	**Sistema**
	1	Clicca "Notifiche" in #M0	
	2		Mostra #M1
	3	Clicca "ACCETTA" o "Rifiuta"	
	4		Mostra #M2 o #M3 e termina UC
Extension A: Cambia finestra	3a	Clicca un'altra icona nel menù in alto a destra	
	4a		Mostra #M0 (relativa al prec.) e termina UC
Extension B: Ricarica la finestra delle notifiche	3b	Clicca l'icona "Notifiche"	
	4b		Ritorna allo step 2


DietiEstates25
La casa che vuoi, quando vuoi
Home
Notifiche
#M0
#M1
Richieste di Visita
Richiedente
alessiamarasco19@gmail.com
Indirizzo immobile
Via Epomeo 73
Data richiesta
3/10/2025, 10:30
Stato
Rifiutata
In attesa
Rifiuta
ACCETTA
#M2
Accettata
#M3

USE CASE#4	Acquirente Richede Visita		
Goal in Context	Un acquirente interessato a visionare un immobile può inviare una richiesta di visita all'agente immobiliare per i successivi sette giorni		
Preconditions	L'acquirente deve aver effettuato i casi d'uso "Eseguire Login", "Effettuare Ricerca" e "Visualizzare Immobile"		
Success End Condition	Il Sistema tiene traccia della richiesta (l'acquirente non potrà richiedere una visita per lo stesso giorno alla stessa ora) e notifica l'agente immobiliare		
Failed End Condition	Il Sistema non tiene traccia della decisione o non notifica l'agente immobiliare		
Primary Actor	Acquirente		
Trigger	Acquirente clicca "RICHIEDI UNA VISITA" in #M0		
Main Scenario	**Step n.**	**Acquirente**	**Sistema**
	1	Clicca "RICHIEDI UNA VISITA" in #M0	
	2		Mostra #M1
	3	Seleziona un giorno e un orario, e clicca "CONFERMA PRENOTAZIONE"	
	4		Attiva il tasto "CONFERMA PRENOTAZIONE"
	5	Clicca "CONFERMA PRENOTAZIONE"	
	6		Mostra #M2
	7	Clicca "PROSEGUI"	
	8		Mostra #M0 e termina UC
Extension A: Clicca "Annulla" in #M1	3a	Clicca "Annulla"	
	4a		Mostra #M0 e termina UC


DietiEstates25
La casa che vuoi, quando vuoi
Home
Notifiche
Cronologia
Preferiti
Lista Annunci
#M0
€ 500.000
Visualizza la mappa
In Vendita in Via Francesco Cilea 83, Napoli, Napoli
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam sed accumsan libero. Morbi consequat venenatis tellus vel viverra. Duis malesuada ex ut nunc sodales, ut porta augue finibus. Proin id ex massa. Integer eget mauris et mi condimentum accumsan. Nulla facilisi. Interdum et malesuada fames ac ante ipsum primis in faucibus. Duis vel purus at lorem rhoncus ornare ut non sapien.
Phasellus quis odio quis libero tempus porta.Suspendisse non ipsum efficitur, volutpat risus quis, semper nibh. Curabitur odio nisi, consequat eget malesuada at, sagittis ut massa. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus ut ligula ornare lorem elementum vehicula sed et sem.
Proin eleifend erat orci, vitae pretium urna convallis eu. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce in tortor sit amet nisi commodo porttitor vitae eu orci. Aliquam non eros eget sem aliquet tincidunt aliquet vitae nulla. Proin nec pretium dui, vel posuere tellus.
Piano: 4
Totale Piani Edificio: 6
Superficie: 130 m²
Numero di Stanze: 4
Classe Energetica: B
Arredamento: Parzialmente Arredato
Riscaldamento: Centralizzato
Stato dell'Immobile: Da Ristrutturare
Ascensore: no
RICHIEDI UNA VISITA
Con un menu a tendina è possibile selezionare un orario che vada dalle 7 alle 19, a scaglioni di 15 minuti
Se l'acquirente ha già richiesto una visita per lo stesso giorno allo stesso orario, quest'ultimo non risulterà disponibile
PRENOTA UNA VISITA
#M1
Scegli una data tra i prossimi 7 giorni e consulta il meteo previsto:
lun 1 soleggiato
mar 2 soleggiato
mer 3 soleggiato
gio 4 soleggiato
ven 5 soleggiato
sab 6 soleggiato
dom 7 soleggiato
Si potrà fare richiesta solo per i sette giorni successivi a quello corrente
Orario: --:--
CONFERMA PRENOTAZIONE
Annulla
Il pulsante "conferma prenotazione" si attiverà solo quando saranno selezionati tutti i dati necessari
#M2
Complimenti!
La richiesta è stata inviata con successo.
Controlla le notifiche per verificare l'esito della richiesta.
PROSEGUI

5 Progettazione Architetturale e Scelte Tecnologiche

5.1 Descrizione dell'Architettura

L'architettura del sistema è stata progettata per garantire modularità, scalabilità e sicurezza, adottando un approccio multilivello che separa nettamente le responsabilità tra frontend e backend. Nello specifico, si fa uso del **pattern MVC** (Model-View-Controller), dove: il Model consiste nelle entità utilizzate nel back-end; il View corrisponde al front-end sviluppato con React (framework usato per la gestione dell'interfaccia utente e l'esperienza visiva del sito). Infine, il Controller coincide proprio con i controller del backend.

Backend: segue una Layered Architecture, organizzata in più livelli:

- I **controller** espongono le API REST e gestiscono la ricezione e la validazione delle richieste. Piu nello specifico, quando ricevono richieste dalla rete, le inviano alle service per l'elaborazione e l'ottenimento di risultati e, infine, inviano le risposte.
- Le **service** contengono la logica di business e orchestrano le operazioni complesse. In particolare, ricevono le richieste dai controller, manipolano le informazioni e le inviano al livello delle repository, così da memorizzarle nel database.
- Le **repository** gestiscono l'accesso ai dati tramite un sistema ORM, sfruttando sia le funzionalità di persistenza standard, sia query personalizzate per esigenze specifiche.

La configurazione del backend comprende moduli orientati alla sicurezza, alla gestione dello storage su cloud e alle policy CORS, fondamentali per consentire l'accesso controllato da parte del frontend. L'autenticazione è gestita tramite **token**, che consente di mantenere la statelessness delle sessioni e di garantire la sicurezza della comunicazione client-server.

Frontend: realizzato come Single Page Application (SPA) con struttura modulare. Ogni funzionalità è incapsulata in un componente dedicato, organizzato in sottocartelle con i relativi file di stile.

L'applicazione utilizza il routing client-side per la navigazione tra le diverse sezioni. La gestione dello stato è eseguita prevalentemente a livello locale nei componenti. Sono presenti componenti presentazionali e container, unitamente all'utilizzo di hook personalizzati per logiche riutilizzabili. L'interfaccia è progettata per essere accessibile e responsive.

L'integrazione con servizi esterni viene effettuata mediante chiamate API asincrone, con gestione di loading, errori e aggiornamenti dello stato dell'interfaccia.

La comunicazione tra frontend e backend avviene esclusivamente tramite API RESTful, con payload in formato JSON. L'integrazione verso uno storage cloud consente la gestione efficiente e sicura di file e media, mentre la configurazione CORS garantisce che solo le richieste autorizzate possano accedere alle risorse protette.

5.2 Criteri di Design e Motivazioni

Il design del sistema si basa su alcuni principi fondamentali dell'ingegneria del software:

- **Separazione delle responsabilità** tra i vari livelli: per mantenere il codice pulito, facilmente testabile e pronto per eventuali evoluzioni future. I controller si occupano esclusivamente di ricevere le richieste e di inviare le risposte, lasciando la logica di business ai service. I repository astraggono l'accesso ai dati, facilitando la modifica della tecnologia sottostante senza toccare la logica applicativa. Questo approccio facilita la manutenzione, i test automatici e l'evoluzione del sistema.

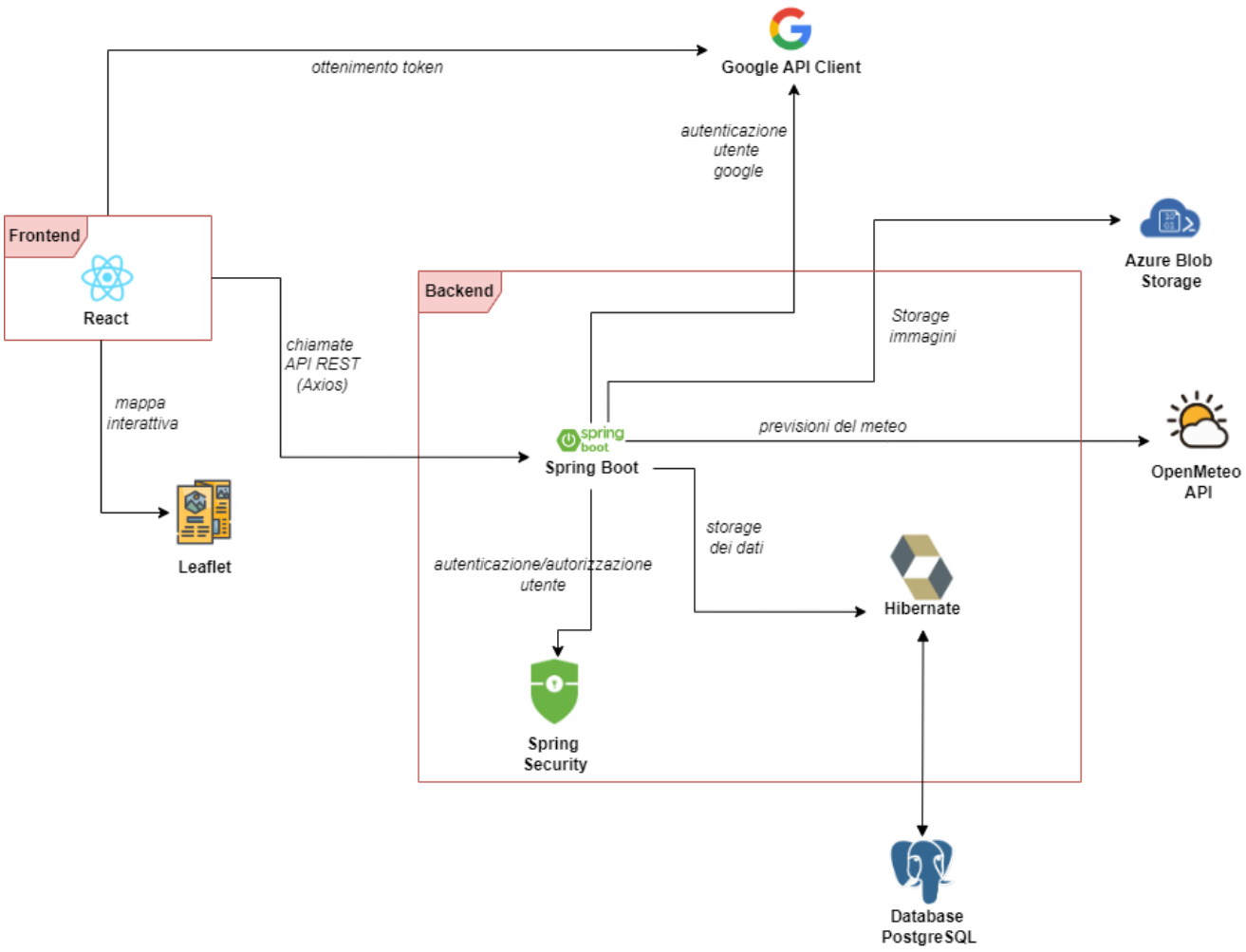
- **Sicurezza:** l'utilizzo di token per l'autenticazione garantisce sessioni sicure e stateless, riducendo i rischi legati alla gestione delle sessioni lato server. La configurazione delle policy di accesso e delle regole CORS protegge da accessi non autorizzati da domini esterni. La scrittura dei file nel cloud storage garantisce affidabilità, scalabilità e sicurezza.

- **Modularità e scalabilità:** la modularità dell'architettura permette di scalare orizzontalmente sia il frontend che il backend. Nel primo, la struttura a componenti consente di aggiungere nuove funzionalità senza impattare il resto dell'applicazione. Nel secondo, l'uso di oggetti di trasferimento dati (DTO) separa i modelli interni da quelli esposti via API, riducendo l'accoppiamento e facilitando eventuali modifiche future.

- **Esperienza utente:** la scelta di una SPA per il frontend permette di offrire un'interfaccia utente reattiva e moderna, con aggiornamenti dinamici della UI senza ricaricare la pagina. Questo migliora l'esperienza utente e rende l'applicazione più performante.

In sintesi, l'architettura e le scelte progettuali adottate sono state guidate dalla volontà di realizzare un sistema robusto, sicuro, facilmente estendibile e in grado di offrire una user experience di alto livello.

5.3 Tecnologie Adottate



Backend:

- **Spring Boot**: framework che semplifica la creazione di applicazioni Java moderne, fornendo strumenti integrati per la gestione di API REST, sicurezza e persistenza dei dati.
- **Spring Security**: modulo per la gestione dell'autenticazione e dell'autorizzazione, fondamentale per proteggere le risorse esposte dal backend.
- **PostgreSQL**: database relazionale open source scelto per la sua affidabilità e scalabilità nella gestione dei dati applicativi.
- **Google API Client**: libreria per l'integrazione dell'autenticazione tramite Google OAuth2/OpenID.
- **Hibernate**: framework ORM che gestisce il mapping tra le entità Java e le tabelle del database, automatizzando operazioni di persistenza, query e gestione delle relazioni.
- **OpenMeteo API**: API utilizzata per ottenre le previsioni del meteo dei successivi 7 giorni.

Frontend:

- **React**: libreria JavaScript per la realizzazione di interfacce utente dinamiche e componentizzate.
- **Vite**: strumento di build e sviluppo rapido che ottimizza il caricamento e la compilazione delle risorse frontend.
- **React Router DOM**: libreria per la gestione del routing client-side e della navigazione tra le diverse sezioni dell'applicazione.
- **Axios**: client HTTP utilizzato per effettuare richieste asincrone verso le API del backend.
- **Leaflet**: libreria per la visualizzazione di mappe interattive all'interno dell'applicazione.
- **React Toastify**: componente per la gestione di notifiche utente non intrusive.

Servizi Cloud e Versionamento:

- **Azure Blob Storage**: servizio cloud utilizzato per l'archiviazione scalabile e sicura di file e media.
- **GitHub**: piattaforma per il versionamento distribuito e la collaborazione sul codice sorgente.

5.4 Schema per la Persistenza dei Dati

Il sistema adotta un modello relazionale per la persistenza dei dati, implementato tramite JPA/Hibernate.

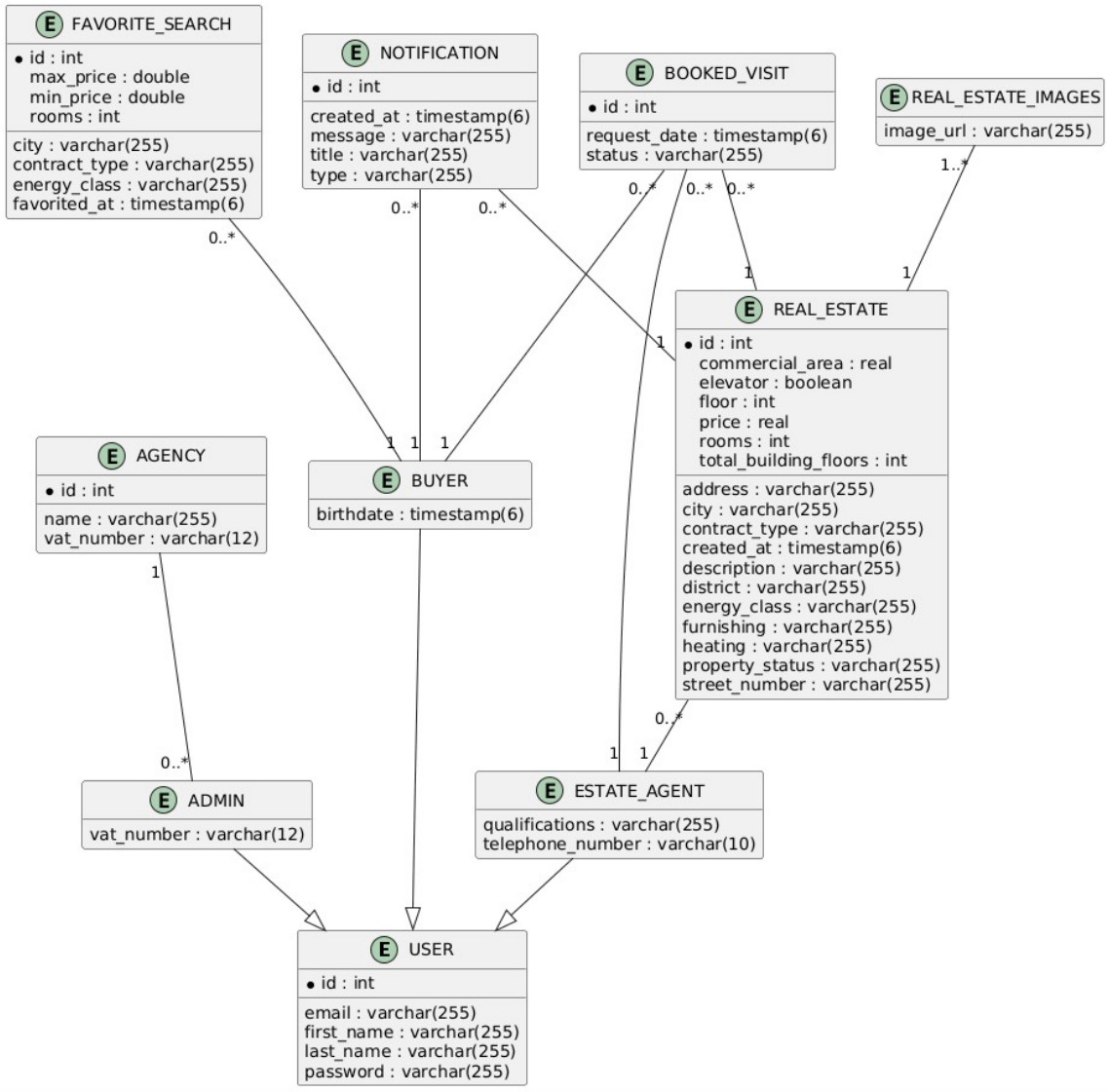


Figura 1: Schema UML delle entità principali e delle relazioni

5.4.1 Descrizione delle entità e delle relazioni

Al centro della struttura dati vi è l'entità User, che rappresenta ogni utente della piattaforma e viene specializzata in Admin, Buyer ed EstateAgent, riflettendo così i diversi ruoli e privilegi.

Ogni agenzia immobiliare è modellata dall'entità Agency, la quale è associata a un amministratore (Admin) che ne gestisce le funzionalità principali.

Gli agenti immobiliari (EstateAgent) possono gestire più immobili (RealEstate), ciascuno dei quali è descritto da attributi come indirizzo, città, prezzo e caratteristiche

tecniche. Gli acquirenti (Buyer) possono salvare ricerche preferite (FavoriteSearch), ricevere notifiche (Notification) e richiedere visite (BookedVisit) agli immobili di interesse. Le visite richieste collegano direttamente acquirente, agente e immobile, rappresentando così un punto di incontro tra domanda e offerta.

Le relazioni tra le entità sono progettate per riflettere i processi reali del dominio: ad esempio, un agente può gestire più immobili, mentre un acquirente può avere molteplici ricerche preferite e prenotare diverse visite. La gestione delle chiavi primarie e delle foreign key è affidata a JPA/Hibernate, che si occupa anche della coerenza e dell'integrità dei dati tramite il supporto alle transazioni. Questo schema garantisce una rappresentazione fedele e flessibile delle dinamiche operative del sistema.

6 Design dell'Interfaccia Utente

6.1 Colori

La palette cromatica dell'applicazione è stata definita a partire dalle tonalità del blu e del bianco, integrate da accenti di verde acqua.

Dal punto di vista psicologico, il blu è tradizionalmente associato a fiducia, affidabilità e competenza, valori centrali nel settore immobiliare, nel quale l'utente deve poter contare sulla piattaforma e sulle informazioni presentate.

Dal punto di vista strategico e comunicativo, la scelta di questi colori richiama il linguaggio visivo di piattaforme come *Immobiliare.it*, *Booking.com* e *Amazon*. Tali aziende, pur operando in settori diversi, condividono una percezione di affidabilità. Adottare una palette affine consente quindi di:

- favorire l'immediatezza di riconoscimento, attraverso un'interfaccia che risulta familiare;
- rafforzare la credibilità del brand attraverso l'associazione con valori già sedimentati nell'immaginario collettivo e accostati a questi colori.

La progettazione dell'interfaccia rispetta inoltre la proporzione 60–30–10, principio ampiamente adottato anche dalle piattaforme già citate e riconosciuto nel design visivo per garantire equilibrio, gerarchia e leggibilità.

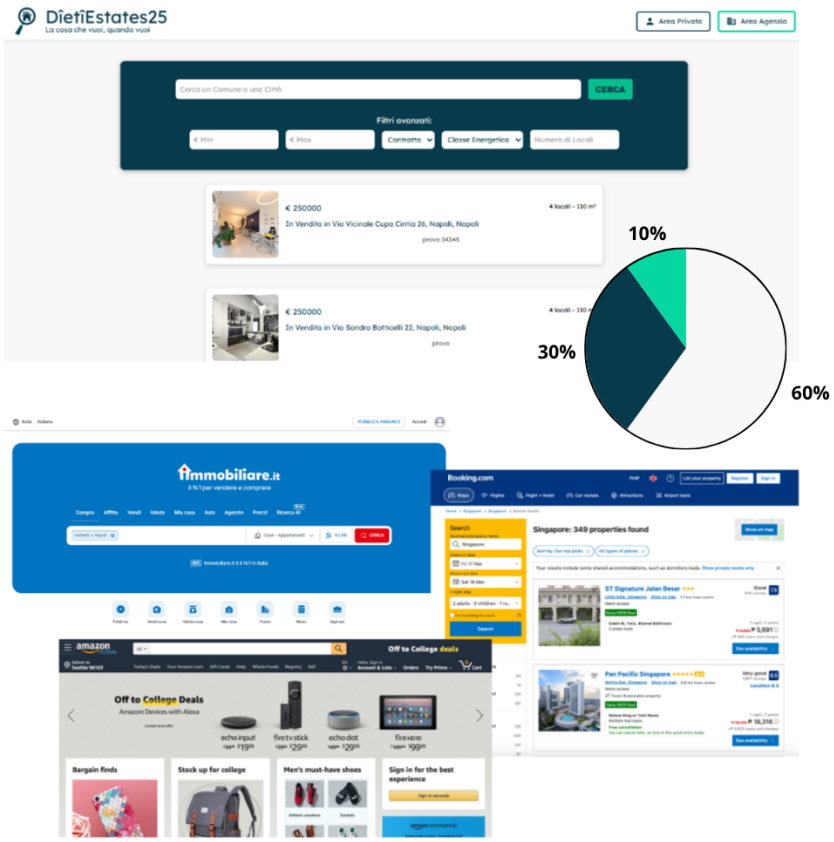


Figura 2: Studio della palette cromatica.

L'utilizzo dell'accent color non è stato semplicemente relegato a pochi dettagli per motivi puramente estetici; ogni elemento è stato invece attentamente ponderato e calcolato.Ad esempio, nella costruzione della HomePage (*Figura 6*), il colore è stato utilizzato per catturare l'attenzione dei due utenti principali sulle funzioni più rilevanti:

- il tasto di ricerca per gli acquirenti;
- l'accesso all'area personale per amministratori e agenti immobiliari.

Questa semplice scelta incrementa notevolmente la gerarchia percepita e cattura immediatamente l'attenzione dell'utente che naviga per la prima volta sulla piattaforma e non ha ancora familiarità con essa.

Particolare attenzione è stata riservata anche al rispetto delle linee guida WCAG, che stabiliscono due livelli di contrasto (**AA** e **AAA**) a cui puntare per garantire la leggibilità dei contenuti.

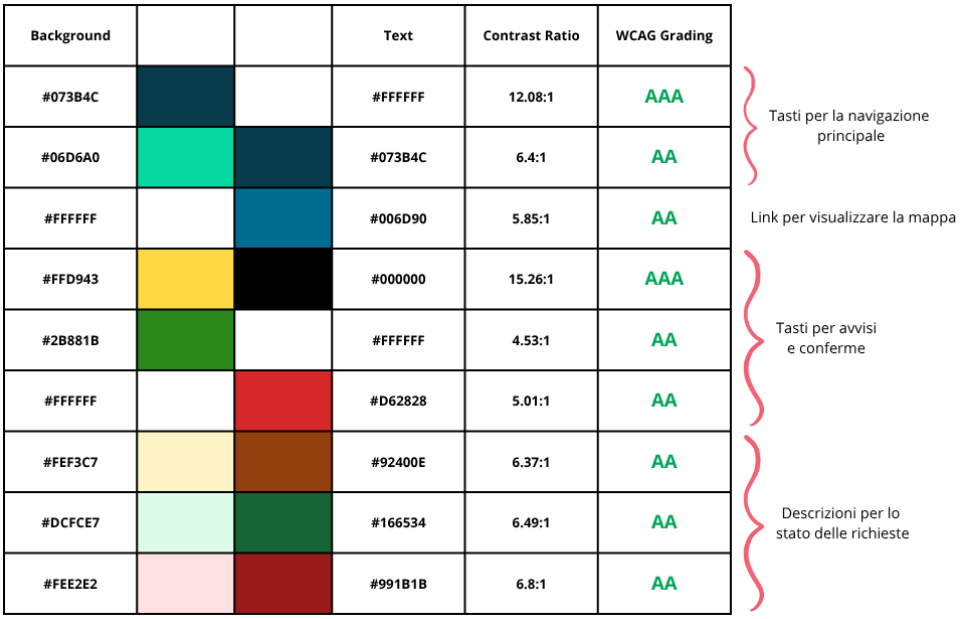


Figura 3: Schema di validazione della palette cromatica in base ai criteri WCAG.

La verifica dei rapporti di contrasto è stata eseguita con strumenti online come *WhoCanUse*. Come si può osservare nello schema, tutte le combinazioni utilizzate in *DietiEstates25* risultano conformi ai requisiti di leggibilità previsti.

Infine, possiamo osservare che la palette non si limita ai colori selezionati come principali per identificare il brand, ma include tonalità che migliorano la comunicazione visiva e la comprensione dei messaggi: il **giallo** identifica gli *avvisi*, il **rosso** segnala gli *errori* e il **verde** i *successi*, secondo una logica coerente e universalmente riconoscibile.

6.2 Tipografia

Per quanto riguarda la tipografia, l'unica scelta degna di nota è stata quella di rappresentare il nome della piattaforma DietiEstates come **DîetîEstates**, inserendo dei piccoli

accenti circonflessi sopra le "i" per simulare dei tetti. Questa scelta stilistica richiama visivamente l'ambito immobiliare della piattaforma, creando un legame immediato tra il brand e la sua funzione principale.

6.3 Labor Perception Bias

Il Labor Perception Bias, ovvero il "bias di percezione del lavoro", è un fenomeno cognitivo per cui gli utenti tendono a considerare un'attività come più significativa o complessa quando viene accompagnata da indicatori visivi di elaborazione, anche se il compito in sé richiede pochissimo tempo o risorse. In altre parole, la presenza di un'attesa o di un'animazione fa percepire il sistema come più scrupoloso e, di conseguenza, aumenta la sensazione di valore del servizio offerto.

Nella piattaforma, questo principio è stato applicato introducendo una finta animazione ogni volta che l'utente clicca sul pulsante di ricerca nella home. L'obiettivo di questa scelta è rassicurare l'utente sul fatto che il sistema sta effettivamente "lavorando" per lui alle massime capacità e rendere l'esperienza più soddisfacente senza introdurre alcun ritardo reale.



Figura 4: Animazione per Labor Perception Bias.

6.4 Principles of Proximity and Connecttedness

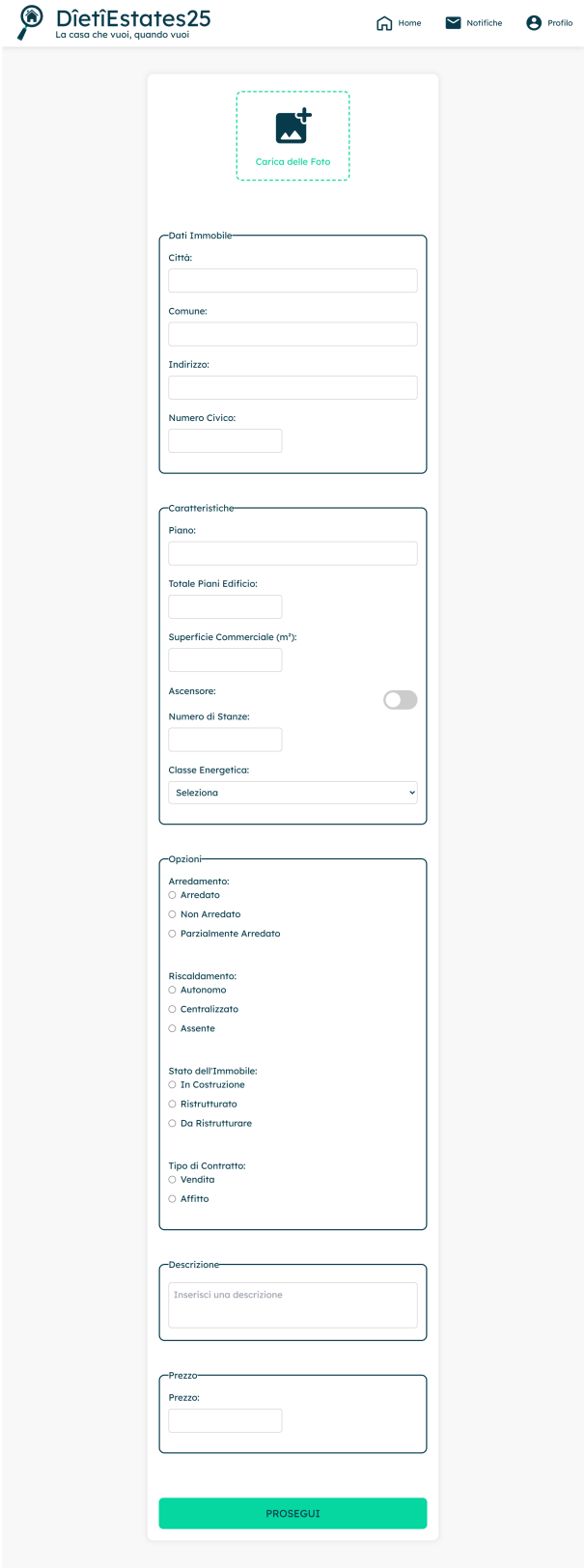
Secondo il principio della prossimità, elementi vicini tra loro saranno percepiti come appartenenti allo stesso gruppo. Il principio di connessione afferma inoltre che elementi visivamente collegati fra loro (con linee, bordi o colori) vengono anch'essi percepiti come appartenenti allo stesso gruppo.

Nell'applicativo di DietiEstates questi principi sono sfruttati in diverse istanze. Un primo esempio è la barra di navigazione:



Nonostante l'icona del profilo non rispetti i canoni delle altre sezioni, viene accomunata a esse dalla vicinanza, dall'utilizzo dello stesso colore e da una regione comune (la barra di navigazione, che si discosta dal resto della pagina come unico elemento bianco).
In questo modo, la navigazione tra le diverse sezioni resta coerente e non si percepisce alcun disequilibrio.

Ancora più lampante è l'utilizzo di questi principi nella pagina dedicata all'aggiunta di un immobile:

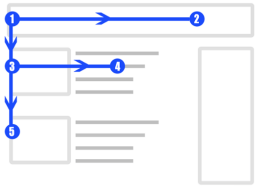


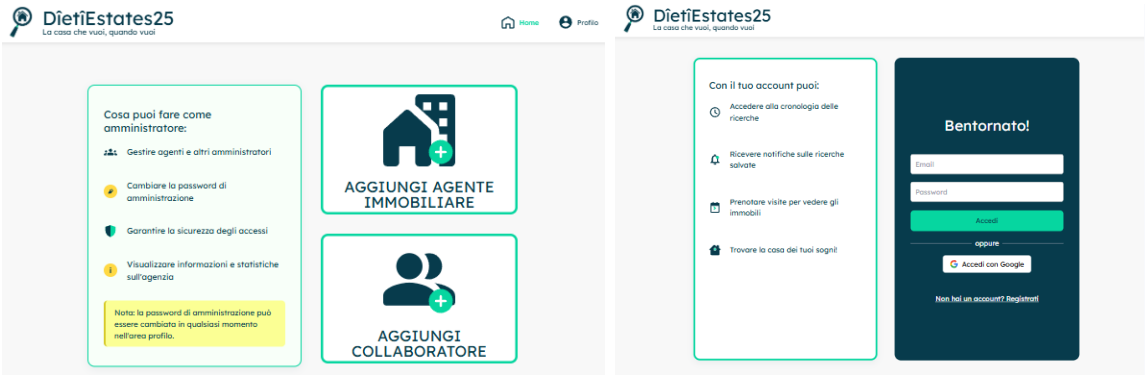
I diversi campi da compilare sono divisi per *argomenti* in aree ben definite e separate tra di loro. In questo modo, non solo l'agente immobiliare non è sopraffatto dai numerosi campi, ma il suo lavoro di compilazione è anche facilitato.

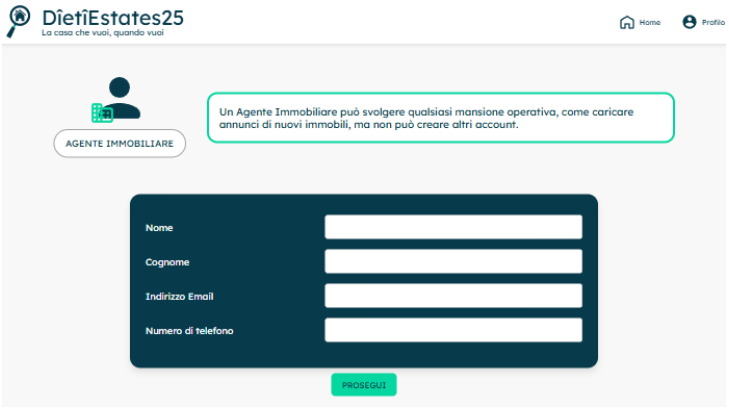
Non è stata scelta una soluzione con più passaggi per dare alla persona che aggiunge l'immobile la possibilità di sapere subito tutte le informazioni di cui ha bisogno per farlo.

6.5 F-Pattern

Nell'organizzare gli elementi all'interno dello schermo è stato tenuto in considerazione l'ordine con cui gli utenti sono natualmente abituati a leggere le informazioni.



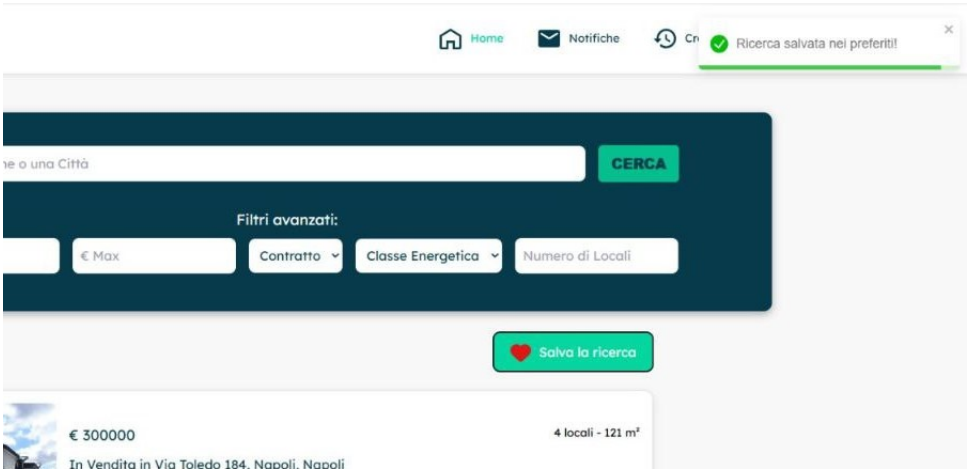




In finestre come quelle precedenti, ad esempio, le diverse sezioni sono state pensate in modo tale che l'utente avesse prima accesso a tutte le informazioni di cui ha potenzialmente bisogno e solo dopo agli elementi con cui interagire.

6.6 Feedback Persistence

Nel comunicare con l'utente, è stata valutata l'importanza dello specifico messaggio caso per caso. Per feedback poco rilevanti, sui quali non è indispensabile che l'utente si concentri, è stato deciso di comunicare attraverso dei toast. Uno di questi compare, ad esempio, per segnalare all'utente quando salva correttamente una ricerca tra i preferiti.



Per comunicazioni più importanti, in cui ci si vuole assicurare che l'utente sia consapevole di ciò che si sta verificando, sono stati scelti dei modal che l'utente è costretto a visualizzare e a cliccare per poter proseguire. Principalmente questa soluzione è propria di avvisi ed errori, ma si è estesa in alcuni casi anche a messaggi di conferma con informazioni cruciali.



6.7 Prevenire gli Errori

È scontato affermare che evitare un errore è più semplice di gestirlo; dove è stato possibile quindi, si è evitato di mettere l'utente in condizione di poterne commettere uno.
Un esempio principe riguarda l'invio della richiesta di visita di un immobile.



In questa interazione l'utente deve selezionare una data, che su richoesta deve essere limitata a uno specifico intervallo di tempo, e un orario, che per logica è stato limitato a un range molto ampio, ma che si limiti a un orario lavorativo.
Per impedire quindi che gli utenti potessero richiedere una visita che non rispettasse questi vincoli, a schermo sono presentati solo i giorni e gli orari idonei.

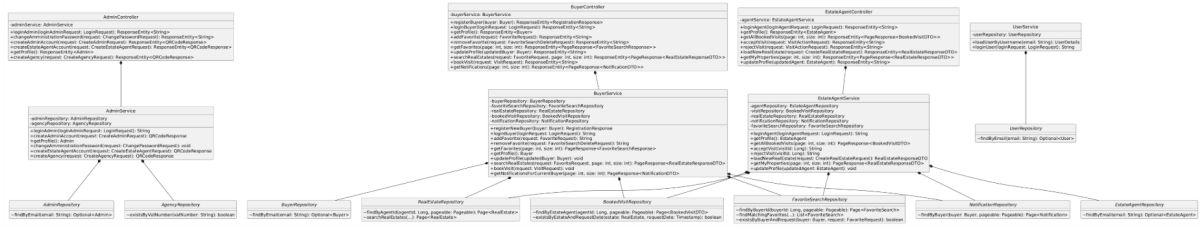
Inoltre, per impedire che un utente sovraccarichi l'agente immobiliare con richieste identiche fra loro, non è data all'utente la possibilità di inviare più richieste per lo stesso giorno con la stessa data. L'orario già selezionato non sarà presente tra quelli disponibili.

7 Diagrammi delle classi di Design

7.1 Classi di Design del Backend

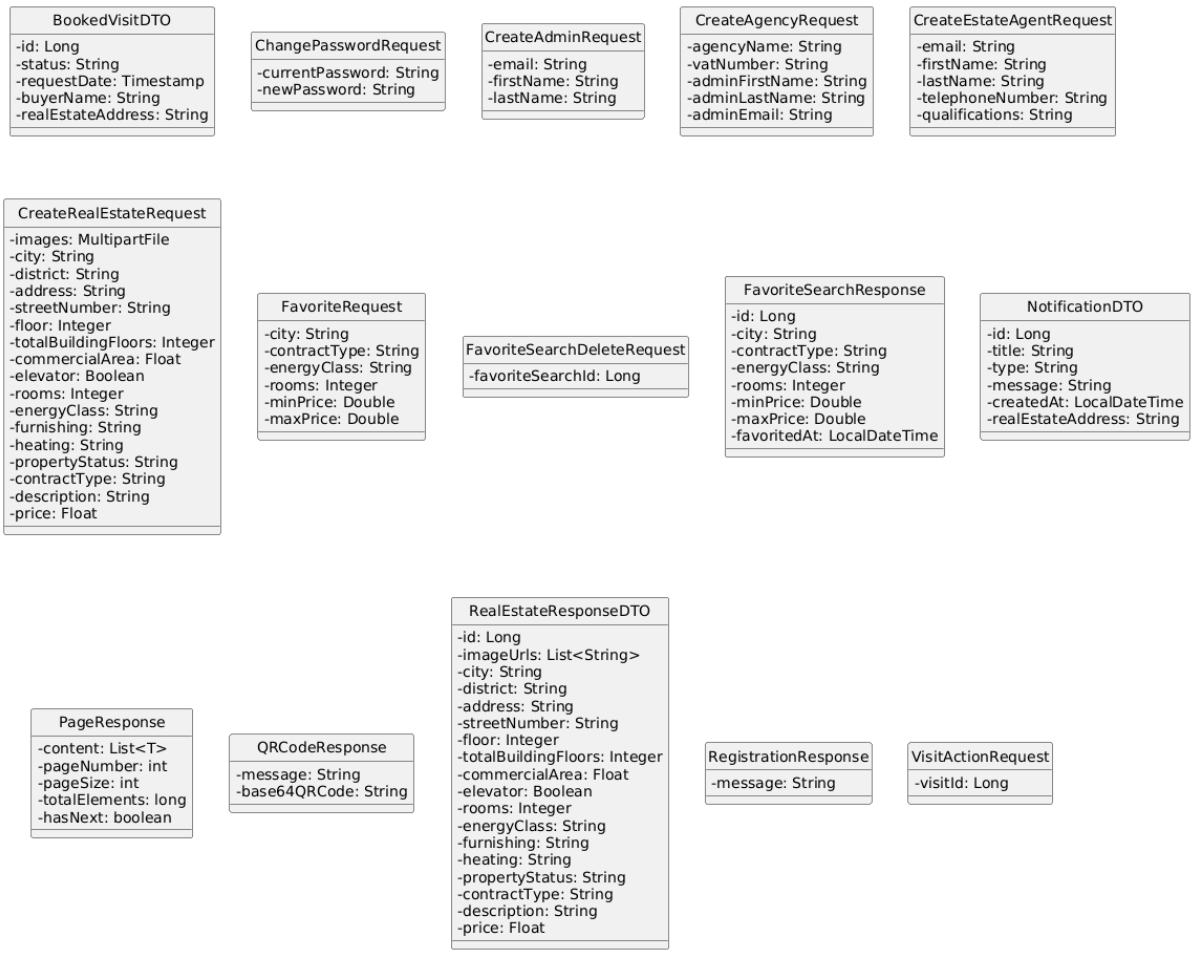
7.1.1 Architettura Controller-Service-Repository

Questo diagramma mostra la struttura architetturale del backend, evidenziando il flusso tra Controller, Service e Repository.



7.1.2 DTO

Questo diagramma mostra tutte le classi DTO utilizzate per il trasferimento dati tra i vari layer dell'applicazione.



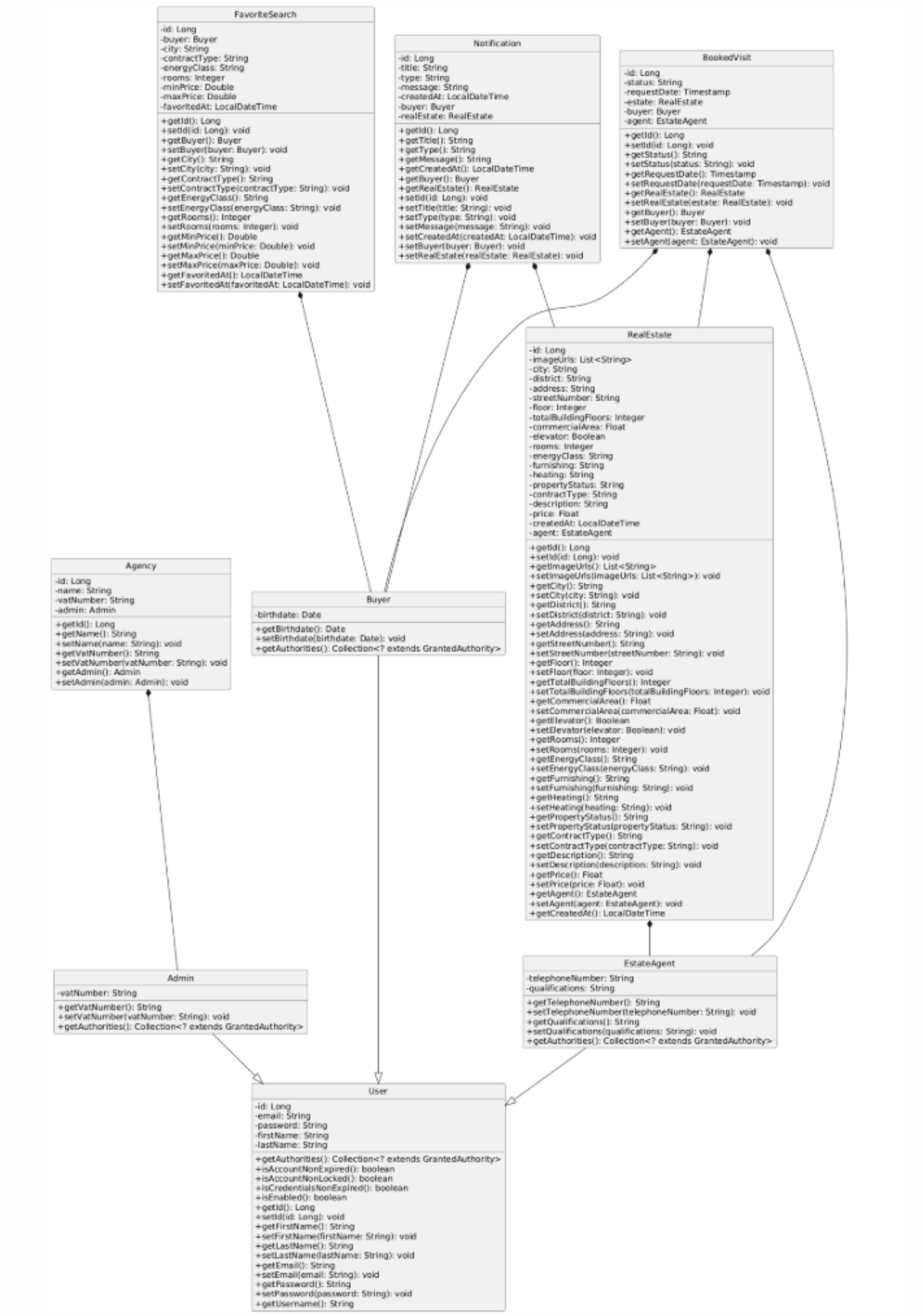
7.1.3 Utility, Config e Security

Il diagramma raggruppa le classi di utilità, configurazione e sicurezza, suddivise per package.



7.1.4 Entità di Dominio (Model)

Il diagramma rappresenta tutte le classi Model del dominio e le relazioni tra di esse.


FavoriteSearch
-id: Long
-buyer: Buyer
-city: String
-contractType: String
-energyClass: String
-rooms: Integer
-minPrice: Double
-maxPrice: Double
-favoritedAt: LocalDateTime
+getId(): Long
+setId(id: Long): void
+getBuyer(): Buyer
+setBuyer(buyer: Buyer): void
+getCity(): String
+setCity(city: String): void
+getContractType(): String
+setContractType(contractType: String): void
+getEnergyClass(): String
+setEnergyClass(energyClass: String): void
+getRooms(): Integer
+setRooms(rooms: Integer): void
+getMinPrice(): Double
+setMinPrice(minPrice: Double): void
+getMaxPrice(): Double
+setMaxPrice(maxPrice: Double): void
+getFavoritedAt(): LocalDateTime
+setFavoritedAt(favoritedAt: LocalDateTime): void
Notification
-id: Long
-title: String
-type: String
-message: String
-createdAt: LocalDateTime
-buyer: Buyer
-realEstate: RealEstate
+getId(): Long
+getTitle(): String
+getType(): String
+getMessage(): String
+getCreatedAt(): LocalDateTime
+getBuyer(): Buyer
+getRealEstate(): RealEstate
+setId(id: Long): void
+setTitle(title: String): void
+setType(type: String): void
+setMessage(message: String): void
+setCreatedAt(createdAt: LocalDateTime): void
+setBuyer(buyer: Buyer): void
+setRealEstate(realEstate: RealEstate): void
BookedVisit
-id: Long
-status: String
-requestDate: Timestamp
-estate: RealEstate
-buyer: Buyer
-agent: EstateAgent
+getId(): Long
+setId(id: Long): void
+getStatus(): String
+setStatus(status: String): void
+getRequestDate(): Timestamp
+setRequestDate(requestDate: Timestamp): void
+getRealEstate(): RealEstate
+setRealEstate(estate: RealEstate): void
+getBuyer(): Buyer
+setBuyer(buyer: Buyer): void
+getAgent(): EstateAgent
+setAgent(agent: EstateAgent): void
RealEstate
-id: Long
-imageUrls: List<String>
-city: String
-district: String
-address: String
-streetNumber: String
-floor: Integer
-totalBuildingFloors: Integer
-commercialArea: Float
-elevator: Boolean
-rooms: Integer
-energyClass: String
-furnishing: String
-heating: String
-propertyStatus: String
-contractType: String
-description: String
-price: Float
-createdAt: LocalDateTime
-agent: EstateAgent
+getId(): Long
+setId(id: Long): void
+getImageUrls(): List<String>
+setImageUrls(imageUrls: List<String>): void
+getCity(): String
+setCity(city: String): void
+getDistrict(): String
+setDistrict(district: String): void
+getAddress(): String
+setAddress(address: String): void
+getStreetNumber(): String
+setStreetNumber(streetNumber: String): void
+getFloor(): Integer
+setFloor(floor: Integer): void
+getTotalBuildingFloors(): Integer
+setTotalBuildingFloors(totalBuildingFloors: Integer): void
+getCommercialArea(): Float
+setCommercialArea(commercialArea: Float): void
+getElevator(): Boolean
+setElevator(elevator: Boolean): void
+getRooms(): Integer
+setRooms(rooms: Integer): void
+getEnergyClass(): String
+setEnergyClass(energyClass: String): void
+getFurnishing(): String
+setFurnishing(furnishing: String): void
+getHeating(): String
+setHeating(heating: String): void
+getPropertyStatus(): String
+setPropertyStatus(propertyStatus: String): void
+getContractType(): String
+setContractType(contractType: String): void
+getDescription(): String
+setDescription(description: String): void
+getPrice(): Float
+setPrice(price: Float): void
+getAgent(): EstateAgent
+setAgent(agent: EstateAgent): void
+getCreatedAt(): LocalDateTime
Agency
-id: Long
-name: String
-vatNumber: String
-admin: Admin
+getId(): Long
+getName(): String
+setName(name: String): void
+getVatNumber(): String
+setVatNumber(vatNumber: String): void
+getAdmin(): Admin
+setAdmin(admin: Admin): void
Buyer
-birthdate: Date
+getBirthdate(): Date
+setBirthdate(birthdate: Date): void
+getAuthorities(): Collection<? extends GrantedAuthority>
Admin
-vatNumber: String
+getVatNumber(): String
+setVatNumber(vatNumber: String): void
+getAuthorities(): Collection<? extends GrantedAuthority>
EstateAgent
-telephoneNumber: String
-qualifications: String
+getTelephoneNumber(): String
+setTelephoneNumber(telephoneNumber: String): void
+getQualifications(): String
+setQualifications(qualifications: String): void
+getAuthorities(): Collection<? extends GrantedAuthority>
User
-id: Long
-email: String
-password: String
-firstName: String
-lastName: String
+getAuthorities(): Collection<? extends GrantedAuthority>
+isAccountNonExpired(): boolean
+isAccountNonLocked(): boolean
+isCredentialsNonExpired(): boolean
+isEnabled(): boolean
+getId(): Long
+setId(id: Long): void
+getFirstName(): String
+setFirstName(firstName: String): void
+getLastName(): String
+setLastName(lastName: String): void
+getEmail(): String
+setEmail(email: String): void
+getPassword(): String
+setPassword(password: String): void
+getUsername(): String

7.2 Classi di Design del Frontend

Questo diagramma delle classi frontend mostra sia le pagine principali che i componenti riutilizzabili dell'applicazione React, evidenziando le relazioni di utilizzo tra di essi. Ogni componente è collegato alle pagine in cui viene impiegato, con i relativi metodi principali, offrendo una panoramica chiara dell'architettura e delle dipendenze interne del frontend. Nel diagramma, le pagine evidenziate in rosso e segnate con il label «Protected» rappresentano le sezioni protette del frontend, accessibili solo agli utenti autenticati.



7.3 Diagrammi di Sequenza di Design per gli Use Case

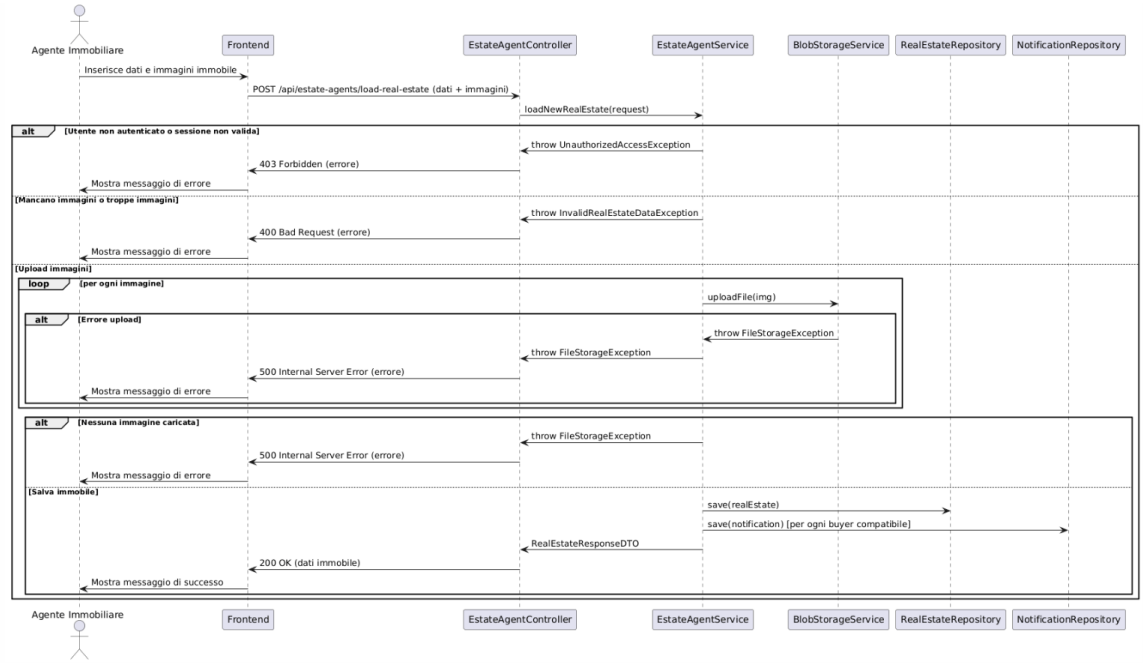


Figura 5: Agente immobiliare carica un nuovo immobile

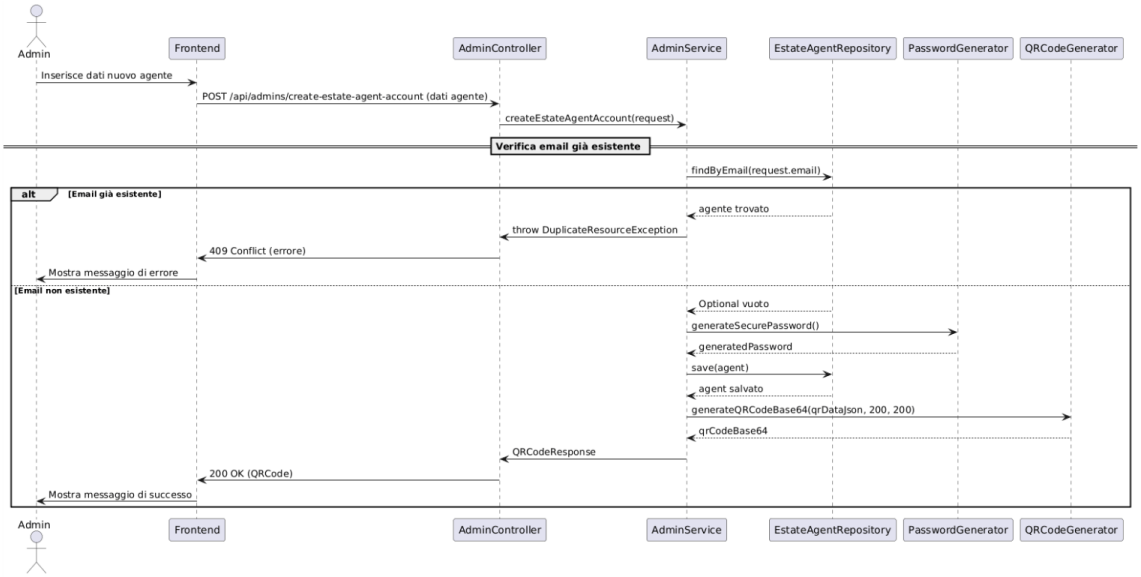


Figura 6: Amministratore aggiunge un nuovo agente agente

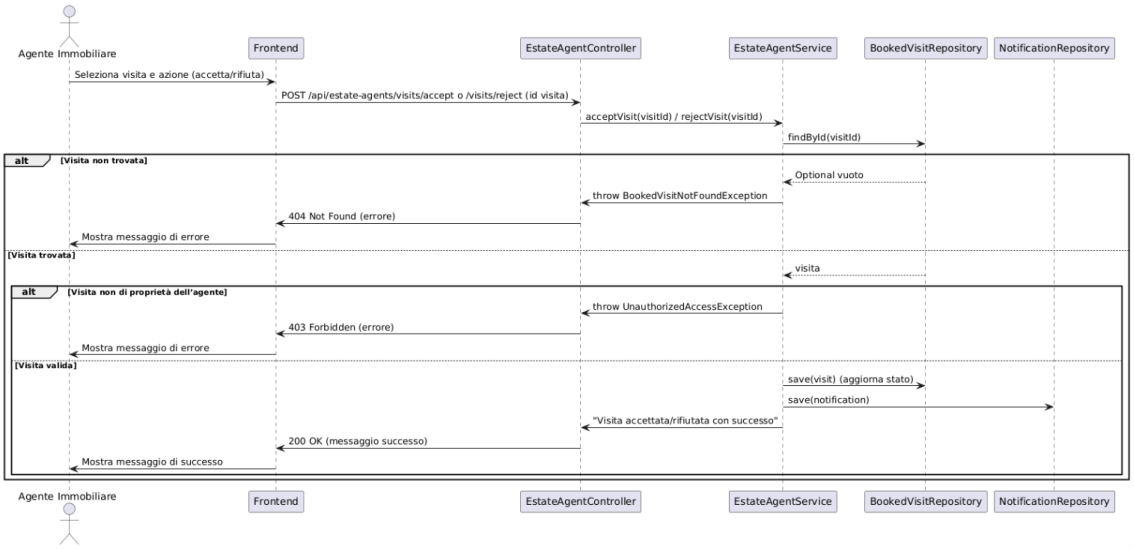


Figura 7: Agente immobiliare gestisce le visite degli immobili

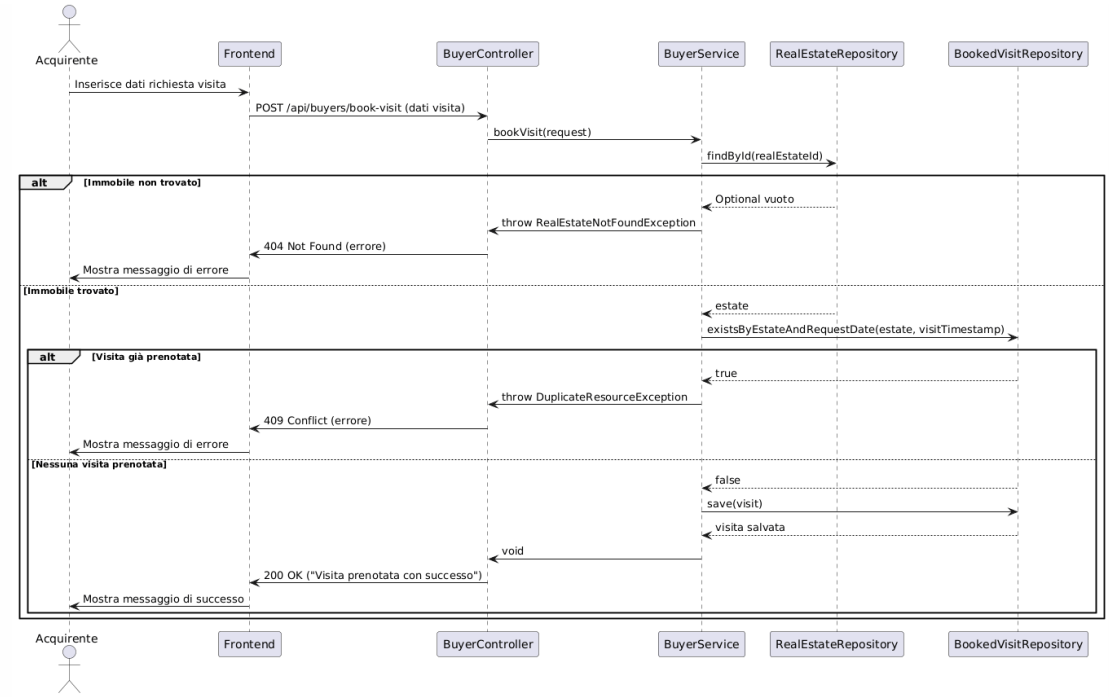


Figura 8: Acquirente richiede una visita per un immobile

8 File di build automatica

Il file di build automatica, basato su Docker e docker-compose, consente di avviare frontend e backend con un solo comando, senza dipendenze locali.

Basta eseguire **docker-compose up** dalla root del progetto per costruire e avviare contemporaneamente i servizi in container isolati, evitando di dover avviare uno per volta i due DockerFile. Ovviamente, è necessario avere avviato Docker Desktop sul proprio pc.

Questa soluzione semplifica la configurazione per tutti i collaboratori e garantisce coerenza tra gli ambienti di sviluppo.

8.1 DockerFile (Backend)

```
FROM maven:3.9.4-eclipse-temurin-21
WORKDIR /app
COPY "BackEnd/Ingegneria_backend/INGSW-BackEnd/" ./
CMD ["mvn", "spring-boot:run"]
```

8.2 DockerFile (Frontend)

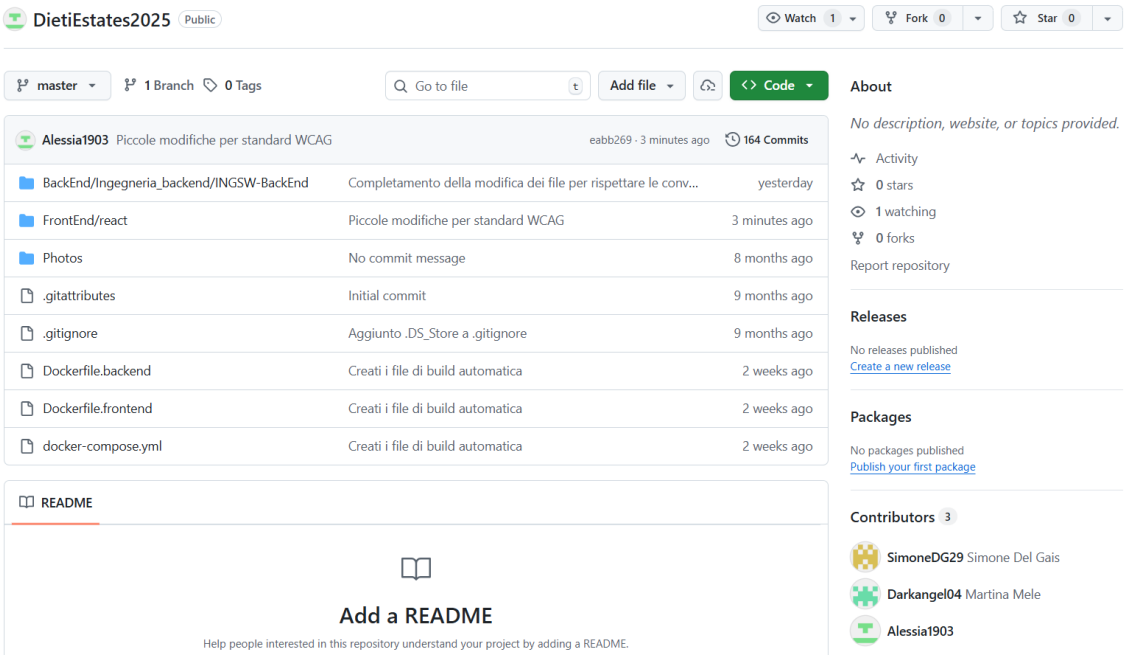
```
FROM node:20
WORKDIR /app
COPY FrontEnd/react/ .
RUN npm install
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

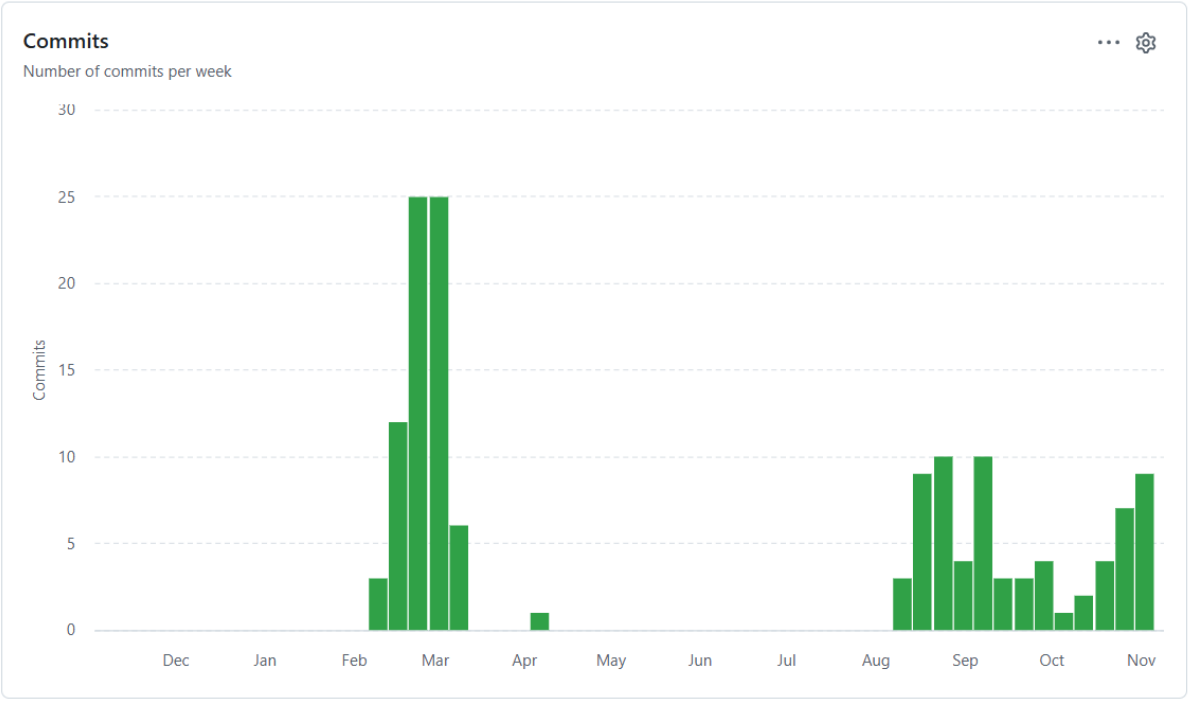
8.3 Docker Compose

```
version: '3'
services:
  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    ports:
      - "5173:5173"
  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    ports:
      - "8080:8080"
```

9 Strumento di Versioning

È stato utilizzato Github come strumento di versioning per la gestione e lo sviluppo del codice. La scelta è stata dettata dalla familiarità del team di sviluppo con lo strumento e le garanzie che esso offre. È stato deciso di utilizzare una singola repository che comprendesse sia il frontend che il backend in modo da semplificare lo sviluppo e la sincronizzazione tra le due parti.



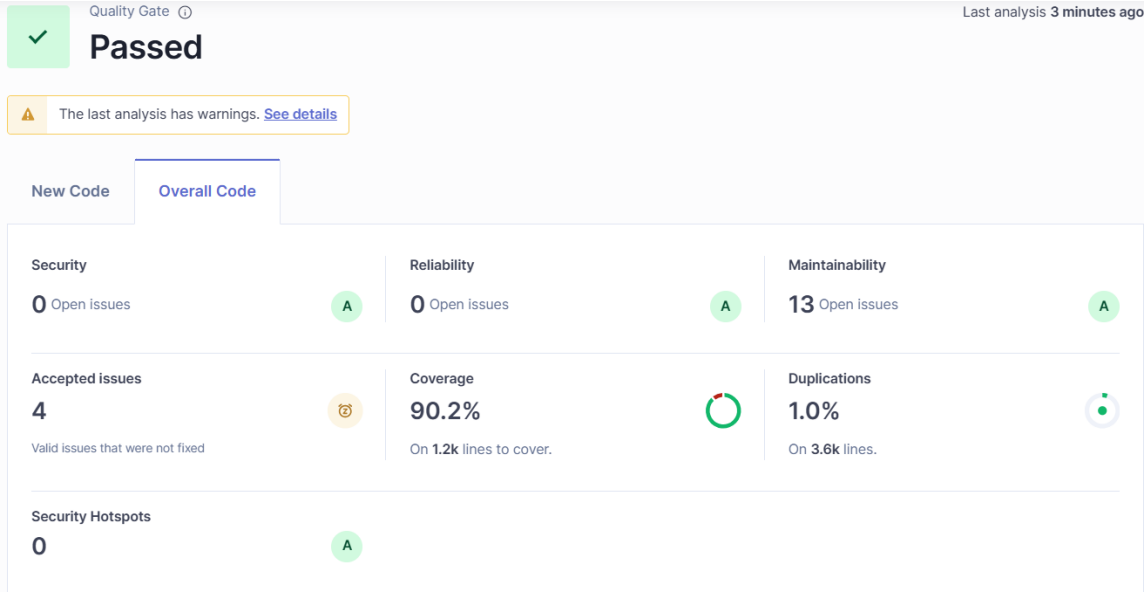


Per ulteriori informazioni, si lascia il link della repository qui sotto:
`https://github.com/Alessia1903/DietiEstates2025`

10 Qualità del codice

È stato utilizzato SonarQube come strumento di analisi del codice per migliorarne la qualità, individuando errori, problemi di sicurezza, code smells, verificando manutenibilità, pulizia del codice e coverage dei test sviluppati.

Con ripetute fasi di risoluzione, è stato superatoe il Quality of Gate di SonarQube e a fornire un sistema quanto più qualitativamente alto possibile.

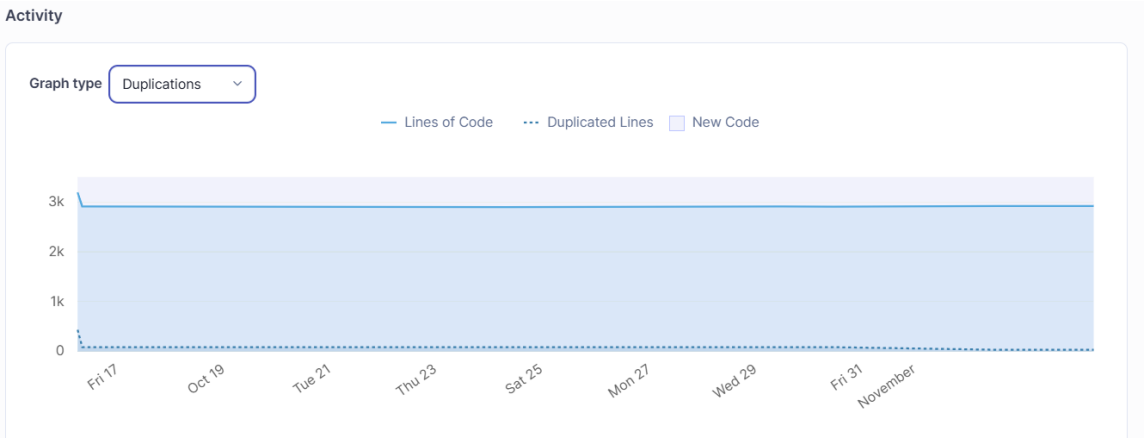


Come si può notare dal report completo, ci sono quattro issues e riguardano l'utilizzo della keyword @MockBean nei test per i controller, considerata una pratica non ottimale. Nonostante ciò, nel report risultano *accettati* per i seguenti motivi.

I test dei controller richiedono la simulazione dell'intero contesto Spring e @MockBean è l'unico strumento fornito da Spring Boot per sostituire bean nel contesto dell'applicazione durante i test.
Le alternative (come creare manualmente i mock) richiederebbero una complessa configurazione del contesto; inoltre si perderebbe l'integrazione automatica con Spring Boot Test.

Invece, con l'approccio adottato, è stato possibile creare test più leggibili e manutenibili, con un setup piu semplice e standardizzato.

10.1 Grafico di Duplicazione del Codice



Durante lo sviluppo, sono state identificate e risolte diverse istanze di duplicazione del codice attraverso due principali strategie:

1. **Utilizzo di Lombok nelle DTO**: una libreria Java che ha permesso di creare getter, setter e costruttori a tempo di compilazione, riducendo significativamente delle linee di codice e ottenendo maggiore manutenibilità.

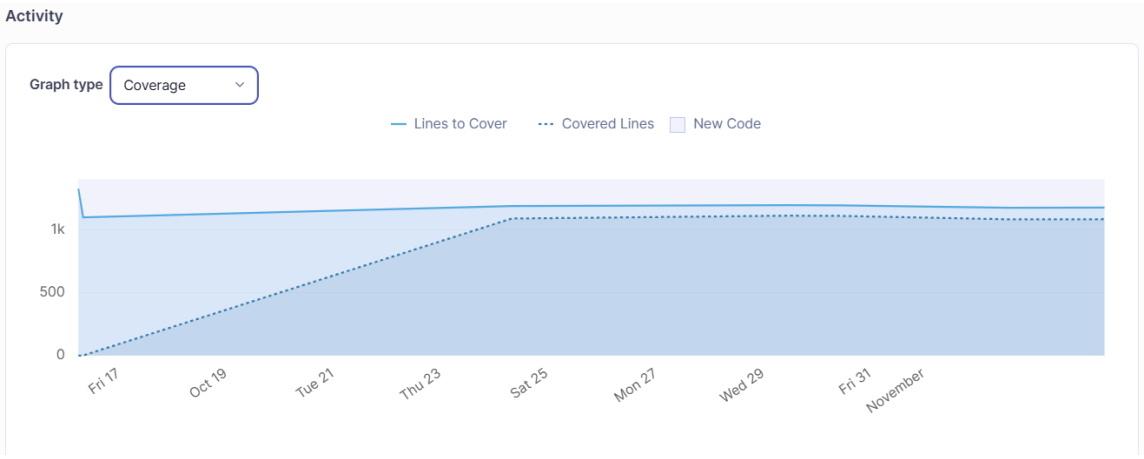
2. **Estrazione di Metodi Comuni**: ove necessario, sono stati creati dei metodi di utilità riutilizzabili nel caso due o più metodi svolgessero uno stesso passaggio, evitando quindi logica duplicata.

10.2 Grafico delle Issues



A causa di uno sviluppo repentino nelle prime fasi, non sono mancati dei code smells, causati principalmente dall'utilizzo di metodi o classi deprecate, import inutilizzati o problemi legati a vulnerabilità e sicurezza generale, prontamente risolti secondo i suggerimenti dati da SonarQube.

10.3 Grafico della Coverage dei Test



Il grafico riporta la coverage dei test, con un miglioramento avvenuto a seguito della copertura di linee di codice segnalate dallo stumento come non coperte. Inoltre è stato utilizzato Jacoco per avere un ulteriore report da confrontare con quello di SonarQube. Le strategie adottate per il testing sono spiegate nella sezione successiva.

11 Unit Testing

11.1 Test di loadNewRealEstate (EstateAgentService)

Il metodo `loadNewRealEstate(CreateRealEstateRequest request)` prende in input un oggetto `CreateRealEstateRequest` e gestisce il caricamento e il salvataggio di un nuovo immobile.

```
public RealEstateResponseDTO loadNewRealEstate(CreateRealEstateRequest request) throws IOException {
    EstateAgent agent = (EstateAgent) SecurityContextHolder.getContext().getAuthentication().getPrincipal();

    if (request.getImages() == null || request.getImages().length == 0)
        throw new InvalidRealEstateDataException("Devi caricare almeno una foto!");
    if (request.getImages().length > 7)
        throw new InvalidRealEstateDataException("Puoi caricare al massimo 7 foto!");

    List<String> imageUrls = new ArrayList<>();
    for (MultipartFile img : request.getImages()) {
        if (img != null && !img.isEmpty()) {
            try {
                String url = blobStorageService.uploadFile(img);
                if (url != null && !url.isEmpty()) imageUrls.add(url);
            } catch (Exception e) {
                throw FileStorageException.uploadFailed(img.getOriginalFilename(), e.getMessage());
            }
        }
    }

    if (imageUrls.isEmpty())
        throw new FileStorageException("Nessuna immagine è stata caricata correttamente");

    RealEstate realEstate = new RealEstate(imageUrls, request.getCity(), request.getDistrict(), request.getAd
    RealEstate savedEstate = realEstateRepository.save(realEstate);

    for (FavoriteSearch fav : favoriteSearchRepository.findMatchingFavorites(savedEstate.getCity(),savedEstat
        notificationRepository.save(new Notification("Nuova proprietà in linea con la tua ricerca", "NUOVA_PR
    return mapToResponseDTO(savedEstate);
}
```

Le principali verifiche effettuate nel codice sono:

- Presenza e numero di immagini;
- Esito dell'upload delle immagini su Blob Storage;
- Salvataggio tramite `realEstateRepository.save()` e creazione di eventuali notifiche.

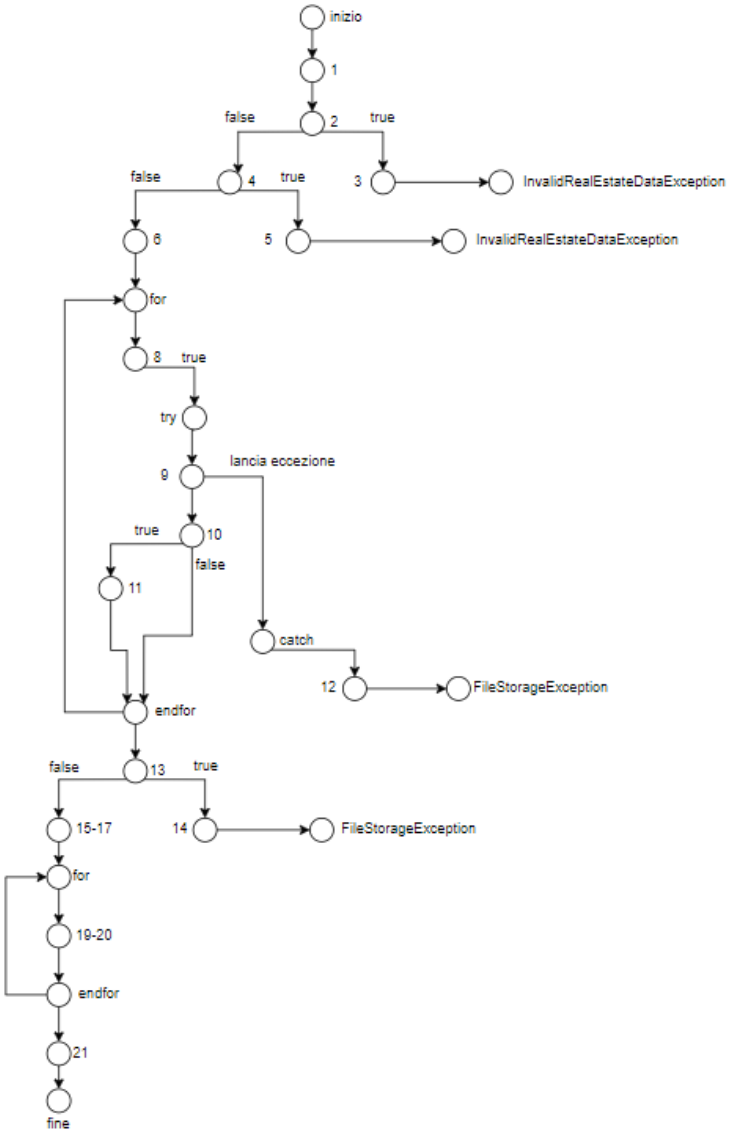


Figura 9: Control Flow Graph - loadNewRealEstate

```
@BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
    SecurityContextHolder.clearContext();
}
```

Figura 10: Setup - loadNewRealEstate

```java
@Test
void testLoadNewRealEstateNoImages() {
    CreateRealEstateRequest req = new CreateRealEstateRequest();
    req.setImages(null);

    SecurityContext context = mock(SecurityContext.class);
    Authentication auth = mock(Authentication.class);
    EstateAgent agent = new EstateAgent();
    when(auth.getPrincipal()).thenReturn(agent);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    assertThrows(InvalidRealEstateDataException.class, () -> estateAgentService.loadNewRealEstate(req));
}
```

Figura 11: TEST 1 (cammino coperto = 1-2-3)

```java
@Test
void testLoadNewRealEstateTooManyImages() {
    CreateRealEstateRequest req = new CreateRealEstateRequest();
    MultipartFile[] images = new MultipartFile[8];
    req.setImages(images);

    SecurityContext context = mock(SecurityContext.class);
    Authentication auth = mock(Authentication.class);
    EstateAgent agent = new EstateAgent();
    when(auth.getPrincipal()).thenReturn(agent);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    assertThrows(InvalidRealEstateDataException.class, () -> estateAgentService.loadNewRealEstate(req));
}
```

Figura 12: TEST 2 (cammino coperto = 1-2-4-5)

```java
@Test
void testLoadNewRealEstateNoImageUploaded() throws IOException {
    EstateAgent agent = new EstateAgent();
    agent.setId(1L);
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(agent);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    CreateRealEstateRequest req = new CreateRealEstateRequest();
    MultipartFile img = mock(MultipartFile.class);
    when(img.isEmpty()).thenReturn(false);
    when(img.getOriginalFilename()).thenReturn(IMAGE);
    MultipartFile[] images = new MultipartFile[]{img};
    req.setImages(images);

    // uploadFile restituisce stringa vuota, quindi nessuna immagine caricata
    when(blobStorageService.uploadFile(any())).thenReturn("");

    assertThrows(FileStorageException.class, () -> estateAgentService.loadNewRealEstate(req));
}
```

Figura 13: TEST 3 (cammino coperto = 1-2-4-6-(8-9-10)*n.foto-13-14)

```java
@Test
void testLoadNewRealEstateUploadFailed() throws IOException {
    EstateAgent agent = new EstateAgent();
    agent.setId(1L);
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(agent);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    CreateRealEstateRequest req = new CreateRealEstateRequest();
    MultipartFile img = mock(MultipartFile.class);
    when(img.isEmpty()).thenReturn(false);
    when(img.getOriginalFilename()).thenReturn(IMAGE);
    MultipartFile[] images = new MultipartFile[]{img};
    req.setImages(images);

    when(blobStorageService.uploadFile(any())).thenThrow(new RuntimeException(message: "upload error"));

    assertThrows(FileStorageException.class, () -> estateAgentService.loadNewRealEstate(req));
}
```

Figura 14: TEST 4 (cammino coperto = 1-2-4-6-8-9-12)

```java
@Test
void testLoadNewRealEstateSendsNotificationsToFavorites() throws IOException {
    EstateAgent agent = new EstateAgent();
    agent.setId(1L);
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(agent);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    CreateRealEstateRequest req = new CreateRealEstateRequest();
    MultipartFile img = mock(MultipartFile.class);
    when(img.isEmpty()).thenReturn(false);
    when(img.getBytes()).thenReturn(new byte[]{1,2,3});
    when(img.getOriginalFilename()).thenReturn(IMAGE);
    MultipartFile[] images = new MultipartFile[]{img};
    req.setImages(images);
    req.setCity(CITY);
    req.setDistrict("Centro");
    req.setAddress(ADDRESS);
    req.setStreetNumber("10");
    req.setFloor(1);
    req.setTotalBuildingFloors(5);
    req.setCommercialArea(100f);
    req.setElevator(true);
    req.setRooms(3);
    req.setEnergyClass("A");
    req.setFurnishing("Arredato");
    req.setHeating("Autonomo");
    req.setPropertyStatus("Nuovo");
    req.setContractType("Vendita");
    req.setDescription("desc");
    req.setPrice(100000f);

    when(blobStorageService.uploadFile(any())).thenReturn(IMAGEURL);
    when(realEstateRepository.save(any())).thenAnswer(invocation -> invocation.getArgument(0));

    FavoriteSearch fav1 = mock(FavoriteSearch.class);
    FavoriteSearch fav2 = mock(FavoriteSearch.class);
    List<FavoriteSearch> favorites = Arrays.asList(fav1, fav2);
    when(favoriteSearchRepository.findMatchingFavorites(any(), any(), any(), any(), anyDouble())).thenReturn(favorites);

    RealEstateResponseDTO resp = estateAgentService.loadNewRealEstate(req);
    assertEquals(CITY, resp.getCity());
    assertEquals(ADDRESS, resp.getAddress());
    assertEquals(1, resp.getImageUrls().size());
    verify(notificationRepository, times(2)).save(any(Notification.class));
}
```

Figura 15: TEST 5 (cammino coperto = 1-2-4-6-(8-9-10-11)*n.foto-13-(15-17)-(19-20)*n.preferiti-21)

```java
@Test
void testLoadNewRealEstateSuccess() throws IOException {
    EstateAgent agent = new EstateAgent();
    agent.setId(1L);
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(agent);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    CreateRealEstateRequest req = new CreateRealEstateRequest();
    MultipartFile img = mock(MultipartFile.class);
    when(img.isEmpty()).thenReturn(false);
    when(img.getBytes()).thenReturn(new byte[]{1,2,3});
    when(img.getOriginalFilename()).thenReturn(IMAGE);
    MultipartFile[] images = new MultipartFile[]{img};
    req.setImages(images);
    req.setCity(CITY);
    req.setDistrict("Centro");
    req.setAddress(ADDRESS);
    req.setStreetNumber("10");
    req.setFloor(1);
    req.setTotalBuildingFloors(5);
    req.setCommercialArea(100f);
    req.setElevator(true);
    req.setRooms(3);
    req.setEnergyClass("A");
    req.setFurnishing("Arredato");
    req.setHeating("Autonomo");
    req.setPropertyStatus("Nuovo");
    req.setContractType("Vendita");
    req.setDescription("desc");
    req.setPrice(100000f);

    when(blobStorageService.uploadFile(any())).thenReturn(IMAGEURL);
    when(realEstateRepository.save(any())).thenAnswer(invocation -> invocation.getArgument(0));
    when(favoriteSearchRepository.findMatchingFavorites(any(), any(), any(), any(), anyDouble())).thenReturn(Collections.emptyList());

    RealEstateResponseDTO resp = estateAgentService.loadNewRealEstate(req);
    assertEquals(CITY, resp.getCity());
    assertEquals(ADDRESS, resp.getAddress());
    assertEquals(1, resp.getImageUrls().size());
}
```

Figura 16: TEST 6 (cammino coperto = 1-2-4-6-(8-9-10-11)*n.foto-13-(15-17)-21)

- Node Coverage: 100% (tutti i nodi/istruzioni sono stati coperti dai test)
- Branch Coverage: 100% (tutti i rami e le condizioni sono stati testati, inclusi i casi di errore)
- Test Effectiveness Ratio: 1 (tutti i percorsi logici sono stati verificati dai test)

11.2 Test di bookVisit (BuyerService)

Il metodo `bookVisit(VisitRequest request)` prende in input un oggetto `VisitRequest` e gestisce la prenotazione di una visita per un immobile. Le principali verifiche effettuate nel codice sono:

- validità della data di visita;
- esistenza dell'immobile;
- disponibilità dell'immobile nella data richiesta.

```java
public void bookVisit(VisitRequest request) {
    Long realEstateId = request.getRealEstateId();
    String date = request.getDate();
    String time = request.getTime();

    RealEstate estate = realEstateRepository.findById(realEstateId).orElseThrow(() -> new RealEstateNotFoundException(

    Buyer buyer = getProfile();
    EstateAgent agent = estate.getAgent();

    String timestamp = String.format("%s %s:00", date, time);
    Timestamp visitTimestamp = Timestamp.valueOf(timestamp);
    boolean exists = bookedVisitRepository.existsByEstateAndRequestDate(estate, visitTimestamp);

    if (exists) {
        throw new DuplicateResourceException("Visita già prenotata per questo immobile, data e orario");}

    BookedVisit visit = new BookedVisit();
    visit.setStatus("In attesa");
    visit.setRequestDate(visitTimestamp);
    visit.setRealEstate(estate);
    visit.setBuyer(buyer);
    visit.setAgent(agent);

    bookedVisitRepository.save(visit);
}
```

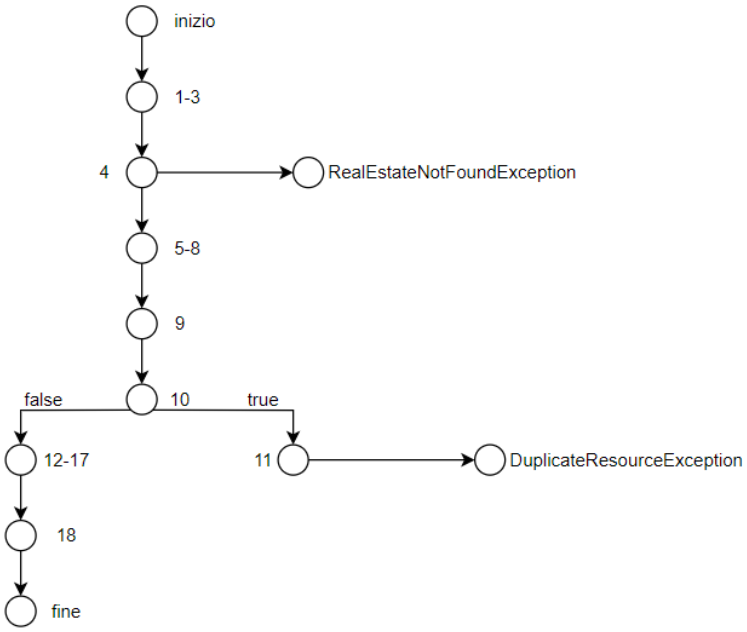


Figura 17: Control Flow Graph - bookVisit

```java
@BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
    SecurityContextHolder.clearContext();
}
```

Figura 18: Setup - bookVisit

```java
@Test
void testBookVisitRealEstateNotFound() {
    VisitRequest req = new VisitRequest();
    req.setRealEstateId(99L);
    req.setDate("2025-11-10");
    req.setTime(TIME);
    when(realEstateRepository.findById(99L)).thenReturn(Optional.empty());
    Buyer buyer = new Buyer();
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(buyer);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    assertThrows(RealEstateNotFoundException.class, () -> buyerService.bookVisit(req));
}
```

Figura 19: TEST 1 (cammino coperto = (1-3)-4-RealEstateNotFoundException)

```java
@Test
void testBookVisitDuplicate() {
    VisitRequest req = new VisitRequest();
    req.setRealEstateId(1L);
    req.setDate("2025-10-10");
    req.setTime(TIME);
    RealEstate estate = new RealEstate();
    estate.setAgent(new EstateAgent());
    when(realEstateRepository.findById(1L)).thenReturn(Optional.of(estate));
    when(bookedVisitRepository.existsByEstateAndRequestDate(any(), any())).thenReturn(true);

    Buyer buyer = new Buyer();
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(buyer);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    assertThrows(DuplicateResourceException.class, () -> buyerService.bookVisit(req));
}
```

Figura 20: TEST 2 (cammino coperto = (1-3)-4-(5-8)-9-10-11)

```java
@Test
void testBookVisitSuccess() {
    VisitRequest req = new VisitRequest();
    req.setRealEstateId(1L);
    req.setDate("2025-10-10");
    req.setTime(TIME);
    RealEstate estate = new RealEstate();
    estate.setAgent(new EstateAgent());
    when(realEstateRepository.findById(1L)).thenReturn(Optional.of(estate));
    when(bookedVisitRepository.existsByEstateAndRequestDate(any(), any())).thenReturn(false);

    Buyer buyer = new Buyer();
    Authentication auth = mock(Authentication.class);
    when(auth.getPrincipal()).thenReturn(buyer);
    SecurityContext context = mock(SecurityContext.class);
    when(context.getAuthentication()).thenReturn(auth);
    SecurityContextHolder.setContext(context);

    assertDoesNotThrow(() -> buyerService.bookVisit(req));
    verify(bookedVisitRepository).save(any(BookedVisit.class));
}
```

Figura 21: TEST 3 (cammino coperto = (1-3)-4-(5-8)-9-10-(12-17)-18)

- - Node Coverage: 100% (tutti i nodi/istruzioni sono stati coperti dai test)
- - Branch Coverage: 100% (tutti i rami e le condizioni sono stati testati, inclusi i casi di errore)
- - Test Effectiveness Ratio: 1 (tutti i percorsi logici sono stati verificati dai test)

11.3 Test di registerBuyerWithGoogle (BuyerService)

Il metodo `registerBuyerWithGoogle(Map<String, String> body)` gestisce la registrazione di un nuovo buyer attraverso Google OAuth2. Le principali verifiche effettuate nel codice sono:

- validazione del token Google;
- verifica dell'unicità dell'email;
- creazione e salvataggio del nuovo buyer.

```java
public RegistrationResponse registerBuyerWithGoogle(Map<String, String> body) {
    String code = body.get("code");
    String redirectUri = body.getOrDefault("redirectUri", "http://localhost:5173/auth/callback");
    String idTokenString = getGoogleIdTokenFromCode(code, redirectUri);
    GoogleIdToken idToken = verifyGoogleIdToken(idTokenString);
    if (idToken == null) {
        throw new InvalidCredentialsException("ID token Google non valido");
    }
    String email = idToken.getPayload().getEmail();
    if (buyerRepository.findByEmail(email).isPresent()) {
        throw new DuplicateResourceException("Buyer già registrato con questa email Google");
    }
    Buyer buyer = new Buyer();
    buyer.setEmail(email);
    buyer.setFirstName((String) idToken.getPayload().get("given_name"));
    buyer.setLastName((String) idToken.getPayload().get("family_name"));
    buyer.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
    buyerRepository.save(buyer);
    return new RegistrationResponse("Registrazione Google andata a buon fine");
}
```

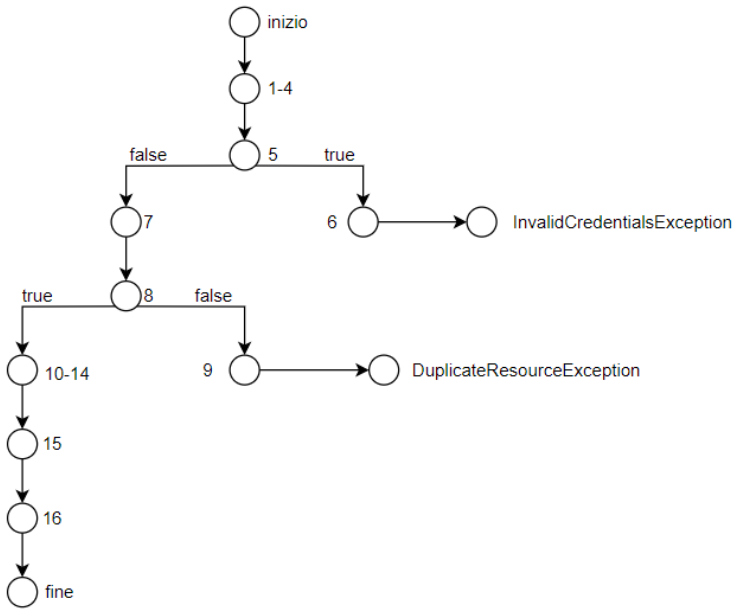


Figura 22: Control Flow Graph - registerBuyerWithGoogle

```java
@BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
    SecurityContextHolder.clearContext();
}
```

Figura 23: Setup - registerBuyerWithGoogle

```java
@Test
void testRegisterBuyerWithGoogleInvalidTokenThrowsException() {
    Map<String, String> body = Map.of(k1: "code", DUMMYCODE);
    doReturn(IDTOKEN).when(buyerService).getGoogleIdTokenFromCode(anyString(), anyString());
    doReturn(null).when(buyerService).verifyGoogleIdToken(anyString());

    assertThrows(InvalidCredentialsException.class, () ->
        buyerService.registerBuyerWithGoogle(body)
    );
}
```

Figura 24: TEST 1 (cammino coperto = (1-4)-5-6)

```java
@Test
void testRegisterBuyerWithGoogleExistingEmailThrowsException() {
    Map<String, String> body = Map.of(k1: "code", DUMMYCODE);
    when(buyerRepository.findByEmail(anyString())).thenReturn(Optional.of(new Buyer()));
    doReturn(IDTOKEN).when(buyerService).getGoogleIdTokenFromCode(anyString(), anyString());
    doReturn(mockValidGoogleIdToken(EMAILTEST)).when(buyerService).verifyGoogleIdToken(anyString());

    assertThrows(DuplicateResourceException.class, () ->
        buyerService.registerBuyerWithGoogle(body)
    );
}
```

Figura 25: TEST 2 (cammino coperto = (1-4)-5-7-8-9)

```java
@Test
void testRegisterBuyerWithGoogleNewUserSuccess() {
    Map<String, String> body = Map.of(k1: "code", DUMMYCODE);
    when(buyerRepository.findByEmail(anyString())).thenReturn(Optional.empty());
    doReturn(IDTOKEN).when(buyerService).getGoogleIdTokenFromCode(anyString(), anyString());
    doReturn(mockValidGoogleIdToken(email: "newuser@example.com")).when(buyerService).verifyGoogleIdToken(anyString());
    when(passwordEncoder.encode(anyString())).thenReturn("encoded");

    RegistrationResponse resp = buyerService.registerBuyerWithGoogle(body);
    assertEquals("Registrazione Google andata a buon fine", resp.getMessage());
    verify(buyerRepository).save(any(Buyer.class));
}
```

Figura 26: TEST 3 (cammino coperto = (1-4)-5-7-8-(10-14)-15-16)

- - Node Coverage: 100% (tutti i nodi/istruzioni sono stati coperti dai test)
- - Branch Coverage: 100% (tutti i rami e le condizioni sono stati testati, inclusi i casi di errore)
- - Test Effectiveness Ratio: 1 (tutti i percorsi logici sono stati verificati dai test)

11.4 Test di getWeatherForecast (WeatherApiService)

Il metodo `getWeatherForecast(String city, String date)` prende in input una città e una data opzionale per recuperare le previsioni meteorologiche.

```
public JsonNode getWeatherForecast(String city, String date) {
    LocalDate startDate = (date != null) ? LocalDate.parse(date) : LocalDate.now();
    LocalDate endDate = startDate.plusDays(6);

    CoordinatesDTO coordinates = getCoordinatesForCity(city);
    if (coordinates == null) {
        throw new WeatherApiException("Coordinate non disponibili per la città: " + city);
    }

    String weatherResult = getWeatherData(coordinates, startDate.toString(), endDate.toString());

    try {
        ObjectMapper mapper = new ObjectMapper();
        JsonNode root = mapper.readTree(weatherResult);
        if (!root.has("daily")) {
            throw new WeatherApiException("Risposta del servizio meteo non valida");
        }
        return root.get("daily");
    } catch (Exception e) {
        throw new WeatherApiException("Errore nell'elaborazione dei dati meteo: " + e.getMessage());
    }
}
```

Le principali verifiche effettuate nel codice sono:

- validità della città (geocoding);
- disponibilità delle coordinate geografiche;
- validità della risposta dell'API meteo.

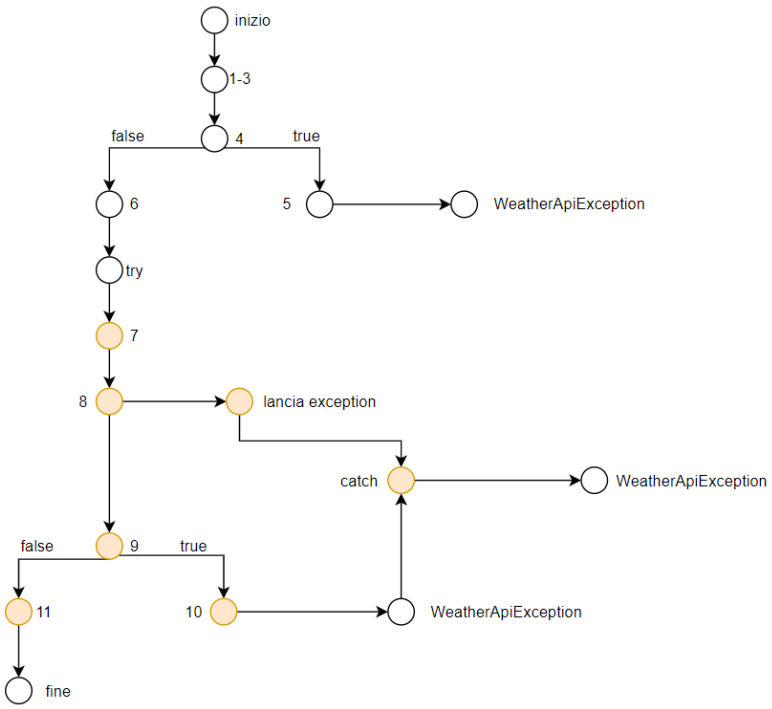


Figura 27: Control Flow Graph - getWeatherForecast
(I nodi segnati in rosso sono nodi contenuti nel try/catch)

```java
@Test
void testGetWeatherForecastCityNotFound() throws IOException {

    doReturn("[]").when(weatherApiService).makeHttpRequest(
        contains(GEO_API_URL),
        eq(USER_AGENT)
    );

    WeatherApiException exception = assertThrows(WeatherApiException.class,
        () -> weatherApiService.getWeatherForecast("CityCheNonEsiste", DATE)
    );
    assertEquals(WEATHER_API_ERROR, exception.getMessage());
}
```

Figura 28: TEST 1 (cammino coperto = ((1-3)-4-5)

```java
@Test
void testGetWeatherForecastInvalidResponse() throws IOException {

    doReturn(GEO_RESPONSE).when(weatherApiService).makeHttpRequest(
        contains(GEO_API_URL),
        eq(USER_AGENT)
    );

    doReturn("{ \"wrong_field\": {} }").when(weatherApiService).makeHttpRequest(
        contains(API_URL),
        isNull()
    );

    WeatherApiException exception = assertThrows(WeatherApiException.class,
        () -> weatherApiService.getWeatherForecast(CITY, DATE)
    );
    assertEquals(WEATHER_API_ERROR, exception.getMessage());
}
```

Figura 29: TEST 2 (cammino coperto = ((1-3)-4-6-7-8-9-10)

```java
@Test
void testGetWeatherForecastSuccess() throws IOException {

    doReturn(GEO_RESPONSE).when(weatherApiService).makeHttpRequest(
        contains(GEO_API_URL),
        eq(USER_AGENT)
    );


    doReturn(EXPECTED_RESPONSE).when(weatherApiService).makeHttpRequest(
        contains(API_URL),
        isNull()
    );

    JsonNode result = weatherApiService.getWeatherForecast(CITY, DATE);

    assertNotNull(result);
    assertTrue(result.has("temperature_2m_max"));
    assertTrue(result.has("temperature_2m_min"));
}
```

Figura 30: TEST 3 (cammino coperto = ((1-3)-4-6-7-8-9-11)

```java
@Test
void testGetWeatherForecastDataProcessingError() throws IOException {
    doReturn(GEO_RESPONSE).when(weatherApiService).makeHttpRequest(
        contains(GEO_API_URL),
        eq(USER_AGENT)
    );

    doReturn("{malformed_json:}").when(weatherApiService).makeHttpRequest(
        contains(API_URL),
        isNull()
    );

    WeatherApiException exception = assertThrows(WeatherApiException.class,
        () -> weatherApiService.getWeatherForecast(CITY, DATE)
    );

    assertEquals(WEATHER_API_ERROR, exception.getMessage());
}
```

Figura 31: TEST 4 (cammino coperto = ((1-3)-4-6-7-8-lancia exception)

- - Node Coverage: 100% (tutti i nodi/istruzioni sono stati coperti dai test)

- - Branch Coverage: 100% (tutti i rami e le condizioni sono stati testati, inclusi i casi di errore)
- - Test Effectiveness Ratio: 1 (tutti i percorsi logici sono stati verificati dai test)

11.5 Strategie adottate

Il progetto usa JUnit 5 come framework di testing principale, seguito poi da Spring Boot Test e Spring Security Test. Per garantire una copertura reale ed efficace delle logiche implementate, abbiamo scelto la strategia White-box come strategia principale per determinare i test necessari.
Utilizzare altre strategie, come R-WECT, si sarebbe rivelato poco efficace, perché la maggior parte delle validazioni sui parametri e dei controlli sui dati viene già effettuata lato frontend. Di conseguenza, applicare R-WECT avrebbe portato a una copertura parziale o addirittura irrilevante dei metodi del backend.

La strategia White-box ha permesso di testare la reale implementazione dei metodi, garantendo quindi la copertura di tutte le logiche, anche quelle non direttamente collegate ai parametri di ingresso. Per ogni test sono stati riportati indicatori della copertura quali Node Coverage, Branch Coverage e Test Effectiveness Ratio. Di seguito sono riportati gli approcci di creazione dei test per le singole sezioni del backend.

11.5.1 Controller

Per il testing dei controller nell'applicazione è stata adottata una strategia completa che sfrutta le potenzialità di Spring Test Framework combinato con MockMvc e Mockito.

La base del testing dei controller si appoggia su MockMvc, uno strumento potente fornito da Spring che permette di simulare richieste HTTP senza la necessità di avviare un server reale. Questo approccio rende i test più veloci e isolati, concentrandosi esclusivamente sulla logica del controller.

Per organizzare i test in modo efficace, è stato seguito un approccio metodico dove ogni endpoint del controller viene testato in diversi scenari. Quindi, sono stati verificati il caso di successo, scenari di errore e accessi non autorizzati. Questo viene fatto simulando diverse condizioni attraverso il mocking del service layer.

La gestione dell'autenticazione e dell'autorizzazione nei test è stata fatta con Spring Security Test, che ci fornisce l'annotazione @WithMockUser, con cui si possono simulare richieste da parte di utenti autenticati con ruoli specifici. Questo è fondamentale per testare endpoint protetti da @PreAuthorize, così da assicurarci che i controller eseguano effettivamente i controlli necessari sulla richiesta.

Per ogni richiesta, verifichiamo non solo che lo status code HTTP sia quello appropriato, ma anche che il contenuto della risposta JSON sia quello atteso. Questo viene fatto utilizzando JsonPath che permette di controllare specifici campi della risposta. Per gli endpoint che richiedono paginazione, come la lista dei preferiti o le notifiche, la verifica si estende anche ai metadata della paginazione, come il numero totale di elementi e la presenza di pagine successive.

Questa strategia di testing completa ci permette di avere un'elevata sicurezza nel corretto funzionamento dei controller, coprendo sia i casi d'uso principali che gli scenari di errore e garantendo che l'API REST si comporti come previsto in tutte le situazioni possibili.

11.5.2 Service

Mentre i controller si concentravano principalmente sulla gestione delle richieste HTTP, qui entriamo nel cuore della logica di business dell'applicazione.

Il testing del service layer richiede una preparazione più elaborata dell'ambiente di test. Prima di ogni test infatti, viene creato un complesso sistema di mock che simula tutte le dipendenze del servizio: i vari repository per l'accesso ai dati, i servizi di supporto (come l'encoder delle password), il servizio JWT (per i token) e il gestore dell'autenticazione. Questo setup permette di isolare completamente il servizio che stiamo testando, garantendo che i test siano veramente unitari.

Dal momento che molte operazioni richiedono un utente autenticato, i test devono simulare questo scenario. Viene quindi fatto un mock dell'intero contesto di sicurezza di Spring, permettendo di simulare diversi stati di autenticazione e autorizzazione. Questo è fondamentale per testare scenari come l'accesso al profilo utente o la gestione dei preferiti.

Inoltre, ci sono anche diverse integrazioni con servizi esterni, come l'autenticazione Google e le API meteo. Testare queste integrazioni richiede un approccio particolare: invece di effettuare chiamate reali ai servizi esterni (che renderebbero i test instabili e dipendenti da fattori esterni), vengono usati dei mock delle risposte HTTP. Questo permette di testare sia gli scenari di successo che quelli di errore in modo controllato e ripetibile.

I test del service layer verificano che le varie eccezioni di business (come credenziali non valide, risorse duplicate o risorse non trovate) vengano lanciate nei momenti appropriati e contengano i messaggi di errore corretti. Questo è cruciale per garantire che l'applicazione si comporti in modo prevedibile anche in situazioni anomale.

Questo approccio completo al testing del service layer garantisce che la logica di business dell'applicazione sia solida e affidabile, gestendo correttamente sia i casi d'uso principali che le varie situazioni di errore che potrebbero verificarsi durante l'utilizzo dell'applicazione.

11.5.3 Repository

Qui il focus è sulla verifica diretta delle interazioni con il database e delle query personalizzate.

La caratteristica più significativa è l'uso dell'annotazione @DataJpaTest, che configura un contesto di test specifico per JPA. Questo setup è particolarmente intelligente perché fornisce automaticamente un database in-memory per i test (chiamato H2), configurando tutto il necessario per testare le operazioni di persistenza senza dover utilizzare un database reale. Questo rende i test veloci, isolati e ripetibili.

La strategia di testing copre diversi scenari chiave. Non ci si limita a testare il caso base di successo, ma si verifica anche il comportamento quando non ci sono risultati, quando ci sono corrispondenze parziali e come funziona la paginazione. Questo approccio completo assicura che le query si comportino correttamente in tutti gli scenari possibili.

Viene utilizzato AssertJ per le asserzioni, rendendo i test più espressivi e leggibili. Invece di semplici assertTrue o assertEquals, vengono utilizzate asserzioni più descrittive come isPresent(), isEmpty(), o isNotEmpty(), che rendono immediatamente chiaro cosa ci si aspetta dal test.

Questa strategia di testing del layer di persistenza garantisce che le operazioni fondamentali di accesso ai dati funzionino correttamente.

11.5.4 Security e Utils

Per quanto riguarda la sicurezza, l'applicazione utilizza un sistema basato su JWT (JSON Web Token) e i test sono strutturati su due livelli: test di integrazione e test unitari del filtro di autenticazione.

Il JwtAuthenticationFilterTest è un test di integrazione che utilizza @WebMvcTest, un'annotazione che configura un contesto Spring minimale per testare i componenti web. Questo test verifica il comportamento del filtro in scenari reali:

- Richieste senza token JWT (deve restituire 401 Unauthorized).
- Richieste con token JWT non valido.
- Richieste con token JWT valido.

Il JwtAuthenticationFilterUnitTest, invece, è un test unitario più dettagliato che si concentra sulla logica interna del filtro. Utilizza la reflection per testare il metodo protetto doFilterInternal e verifica accuratamente:

- La corretta estrazione delle credenziali dal token.
- Il caricamento dell'utente tramite UserDetailsService.
- La validazione del token.
- La corretta impostazione del SecurityContext.

Un aspetto interessante è l'uso di MockHttpServletRequest e MockHttpServletResponse per simulare richieste HTTP senza dover avviare un server web reale, rendendo i test più veloci e isolati.

Passando alle utility, troviamo due classi di supporto con i loro rispettivi test: PasswordGenerator e QRCodeGenerator.

I test del PasswordGenerator sono un ottimo esempio di come testare funzionalità che coinvolgono la generazione di valori casuali. Invece di testare il valore specifico della password generata (che sarebbe impossibile dato che è casuale), i test verificano le proprietà che la password deve rispettare:

- La lunghezza corretta (12 caratteri).

- La randomicità (due password generate consecutivamente devono essere diverse).
- Non può essere null o vuota.

I test del QRCodeGenerator mostrano invece come gestire componenti che interagiscono con librerie esterne (in questo caso zxing per la generazione di codici QR). I test verificano:

- La generazione di QR code come array di byte.
- La generazione di QR code come stringa Base64.
- La gestione appropriata degli errori con input invalidi.

In entrambi i casi, i test sono scritti in modo da essere deterministici e ripetibili, anche quando si tratta di funzionalità che coinvolgono elementi casuali o dipendenze esterne.

12 Valutazione dell'Usabilità

12.1 Expert Review

In questa fase sono stati coinvolti cinque esperti per la valutazione del sistema attraverso una checklist conforme alle otto regole d'oro di Shneiderman.

- Mantieni la coerenza;
- Fornisci scorciatoie per utenti esperti;
- Fornisci un feedback informativo;
- Progetta dialoghi che raggiungano una chiusura;
- Previeni e gestisci gli errori;
- Permetti l'annullamento e il recupero;
- Supporta il controllo dell'utente;
- Riduci il carico di memoria a breve termine;

12.1.1 Obiettivo del Test

Valutare l'usabilità complessiva del sistema, identificando punti di forza e criticità in relazione ai bisogni di ricerca, pubblicazione e gestione degli annunci, secondo le otto regole d'oro di Shneiderman.

12.1.2 Istruzioni per l'Uso della Checklist

1. Ogni utente assegna un voto da 1 a 5 per ciascun principio.
    - 1 = molto scarso
    - 2 = scarso
    - 3 = sufficiente
    - 4 = buono
    - 5 = ottimo
2. Viene calcolata la media per ciascun principio e poi la media totale complessiva.
3. Analisi dei risultati:
    - Punteggio 4–5 = principio rispettato completamente o quasi.
    - Punteggio 3 = principio parzialmente rispettato. Da rivedere in alcune parti.
    - Punteggio 1–2 = principio non rispettato. Necessaria risoluzione immediata.

12.1.3 Risultati

Regola di Shneiderman	**U1**	**U2**	**U3**	**U4**	**U5**	**Media**
Mantieni la coerenza	4	5	4	4	5	4.4
Fornisci scorciatoie per utenti esperti	3	3	4	3	4	3.4
Fornisci un feedback informativo	5	4	5	5	4	4.6
Progetta dialoghi che raggiungano una chiusura	4	4	5	4	4	4.2
Previeni e gestisci gli errori	4	3	4	4	4	3.8
Permetti l'annullamento e il recupero	3	4	3	4	3	3.4
Supporta il controllo dell'utente	5	4	5	4	5	4.6
Riduci il carico di memoria a breve termine	4	5	4	5	4	4.4
Media complessiva	4.0	4.0	4.25	4.125	4.125	**4.1**

12.2 Relazione Sperimentale

12.2.1 Descrizione dei Partecipanti

Sono stati coinvolti **cinque partecipanti** nella valutazione sperimentale. I profili sono stati selezionati per rappresentare le diverse tipologie di utenti che potrebbero utilizzare un sito di immobiliare, con differenti livelli di esperienza digitale e obiettivi (acquisto, affitto, gestione annunci).

ID	Età	Impiego	Esperienza informatica	Uso delle tecnologie	Uso di siti immobiliari
U1	23	Studente universitario	Media-Alta	Utilizza regolarmente piattaforme digitali per studio e svago.	No
U2	26	Ingegnere elettronico	Alta	Usa quotidianamente strumenti digitali avanzati per lavoro.	Sì
U3	26	Pasticciera	Media	Utilizza principalmente smartphone e social media.	Sì
U4	49	Tecnico specializzato	Media-Alta	Lavora quotidianamente con strumenti digitali.	Sì
U5	59	Presepista	Bassa	Utilizza lo smartphone per comunicare e informarsi; usa il computer solo occasionalmente.	Sì

12.2.2 Procedura Sperimentale

Ogni utente ha eseguito individualmente una sessione di test della durata di circa 20–25 minuti, in presenza di un osservatore. Durante la sessione, gli utenti sono stati invitati a pensare ad alta voce e sono state registrate osservazioni su difficoltà, errori e tempi di completamento.

In qualità di utenti interessati alla ricerca di immobili, ai soggetti sperimentali è stato chiesto di:

- Effettuare la registrazione.
- Effettuare l'accesso al portale.
- Cercare un appartamento in affitto in una determinata zona con specifici filtri (es. prezzo, città, numero locali).
- Prenotare una visita per un immobile.

In qualità di agenti immobiliari, ai soggetti sperimentali è stato chiesto di:

- Effettuare l'accesso al portale.
- Pubblicare un nuovo annuncio immobiliare con foto e descrizione.
- Accettare la richiesta di una visita.

In qualità di amministratori, ai soggetti sperimentali è stato chiesto di:

- Effettuare l'accesso al portale.
- Modificare la password di amministrazione.
- Aggiungere un agente immobiliare.

12.2.3 Dati Raccolti

Per ciascun task e per ciascun utente sono stati raccolti i seguenti parametri:

- **Tempo di completamento (s)**: tempo impiegato per portare a termine il task.
- **Tasso di successo (%)**: percentuale di utenti che hanno completato il task senza errori.
- **Errori osservati**: numero e tipo di errori commessi.
- **Soddisfazione soggettiva**: valutazione post-task su scala 1–5.

Tempi di completamento per task Utente Privato (in secondi)

	U1	U2	U3	U4	U5	Tasso di Successo
REGISTRAZIONE	40	40	51	55	82	**100%**
ACCESSO	25	24	30	24	25	**100%**
RICERCA	34	28	30	29	30	**100%**
PRENOTAZIONE	51	45	55	50	47	**100%**

L'unico errore avvenuto è stato commesso da U5: alla richiesta di effettuare la registrazione non ha proseguito col tasto "Non hai un account? Registrati!" ma ha usato i campi di email e password presenti per l'accesso. Nonostatnte ciò, si è autonomamente accorta dell'errore e ha portato a termine la task con successo.

Tutti i soggetti hanno prestato attenzione al modale di conferma della richiesta di prenotazione, comprendendo che non si trattasse di una prenotazione, ma di una richiesta che doveva attendere un esito.

Tempi di completamento per task Agente Immobiliare (in secondi)

	U1	U2	U3	U4	U5	Tasso di Successo
ACCESSO	35	26	28	24	24	**100%**
ANNUNCIO	270	262	269	278	299	**100%**
RICHIESTA	46	44	44	50	47	**100%**

Tutti i soggetti si sono rivolti senza esitazione all' "Area Agenzia" e hanno prestato attenzione ai modali di avviso.

Tempi di completamento per task Amministratore (in secondi)

	U1	U2	U3	U4	U5	Tasso di Successo
ACCESSO	25	23	25	24	24	**100%**
PASSWORD	27	26	36	25	28	**100%**
AGENTE	46	44	44	52	63	**100%**

Tutti i soggetti hanno prestato attenzione ai modali di avviso/conferma. Nello specifico, si è rivelata particolarmente efficace la nota presente nella home sul cambio delle credenziali. Il soggetto U3, inizialmente dubbioso, ha autonomamente letto la nota e completato la task con successo.

12.2.4 Valutazione post-task

Come specificato precedentemente, per il punto "Soddisfazione soggettiva" nelle metriche di valutazione, i soggetti sperimentali hanno valutato il sistema attraverso un survey che ha permesso al team di sviluppo di avere una panoramica sulla soddisfazione nell'usare il sistema.

Ai soggetti sperimentali è stato richiesto di valutare, con un parametro crescente di apprezzamento da 1 a 5, il livello di soddisfazione per le task eseguite.

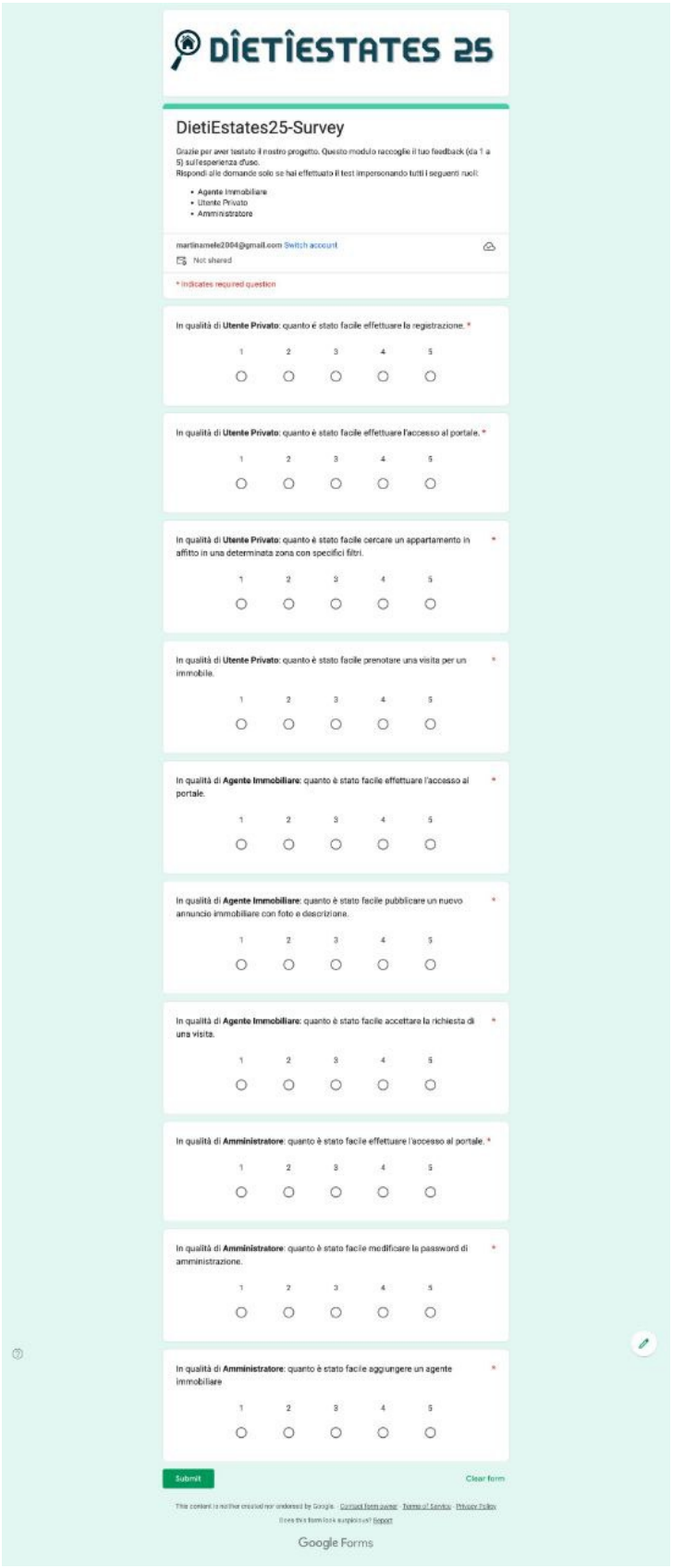
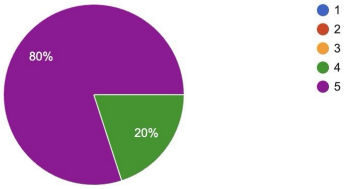


Figura 32: Survey proposto ai soggetti sperimentali

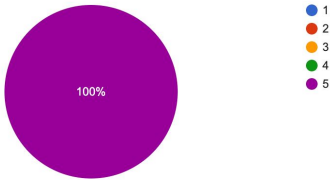
In qualità di **Utente Privato**: quanto é stato facile effettuare la registrazione.

5 responses



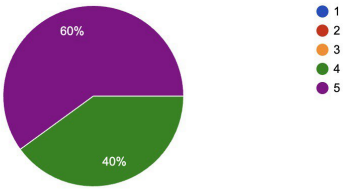
In qualità di **Utente Privato**: quanto è stato facile effettuare l'accesso al portale.

5 responses



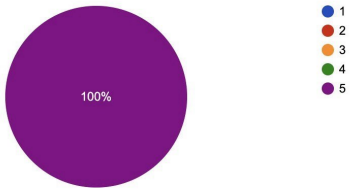
In qualità di **Utente Privato**: quanto è stato facile cercare un appartamento in affitto in una determinata zona con specifici filtri.

5 responses



In qualità di **Utente Privato**: quanto è stato facile prenotare una visita per un immobile.

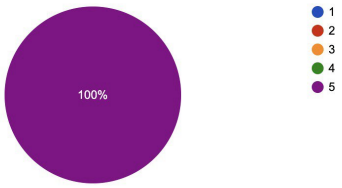
5 responses



- Il soggetto U4 ha sottolineato il suo apprezzamento per il modello di input selezionato per il campo "data di nascita" nella registrazione.
- Alcuni utenti hanno suggerito di aggiungere ulteriori opzioni di filtraggio nella ricerca.
- Tutti i soggetti, eccetto U2, hanno spontaneamente apprezzatto la realizzazione del modale di visualizzazione del meteo nell'effettuare una prenotazione.
- Tutti i soggetti sono stati colpiti positivamente dalla schermata di caricamento nella ricerca.

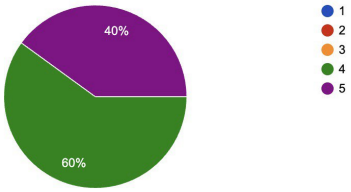
In qualità di **Agente Immobiliare**: quanto è stato facile effettuare l'accesso al portale.

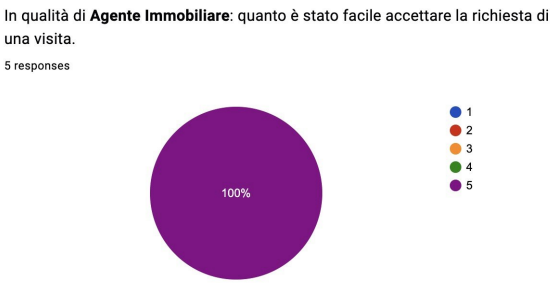
5 responses



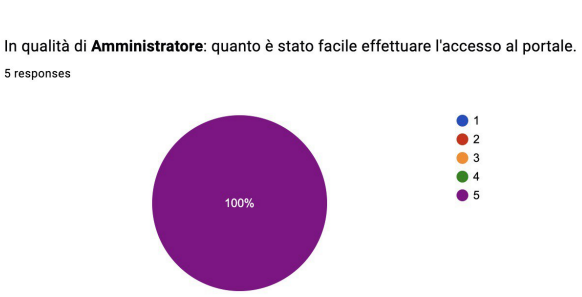
In qualità di **Agente Immobiliare**: quanto è stato facile pubblicare un nuovo annuncio immobiliare con foto e descrizione.

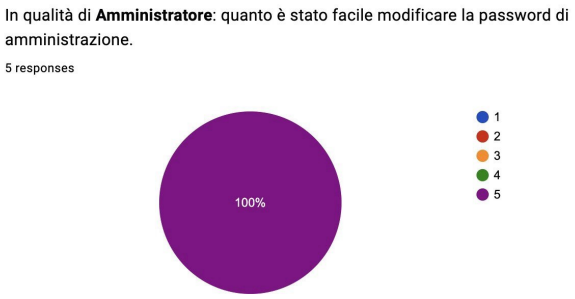
5 responses

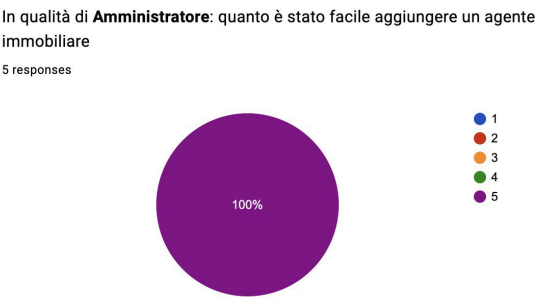




- Alcuni utenti hanno segnalato incertezze nel caricamento delle foto o nella visualizzazione dell'anteprima. Tuttavia, la procedura è stata giudicata sufficientemente chiara.







- La nota fornita per indirizzare correttamente gli utenti alla sezione profilo per il cambio della password si è rivelata particolarmente utile per uno dei soggetti, inizialmente perplesso dalla richiesta.

12.2.5 Conclusioni

I risultati del survey mostrano un livello complessivo di usabilità elevato. La maggior parte dei soggetti sperimentali ha valutato positivamente la semplicità delle principali operazioni, con punteggi medi superiori a 4 su 5.

Questa valutazione ci ha anche permesso di verificare che uno dei nostri obiettivi principali sull'usabilità fosse stato raggiunto: essendo gli amministratori la tipologia di utente che più raramente utilizza l'applicativo, era indispensabile che proprio questa parte avesse la curva di apprendimento meno ripida possibile.

Grazie ai dati raccolti, abbiamo avuto modo di verificare che, nel complesso, tra i tre gruppi di task, quelle degli amministratori sono quelle che meno generano errori e/o confusione. In più, tutti gli aiuti visivi che erano previsti sono stati utilizzati e hanno effettivamente agevolato i potenziali utenti del sistema.